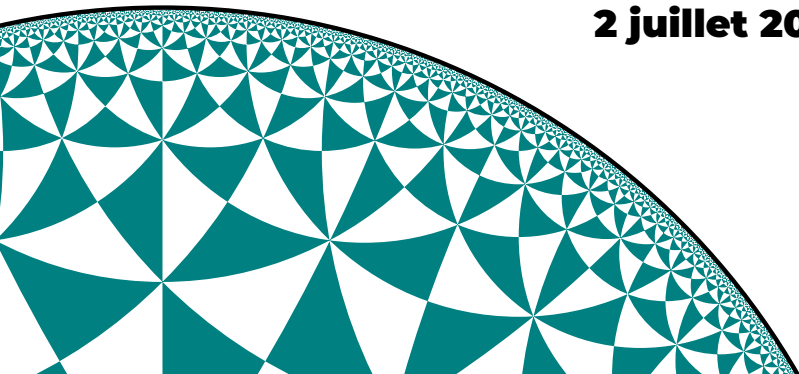


Géométrie hyperbolique avec le package luahyperbolic

Damien Mégy (Université de Lorraine, IECL)

2 juillet 2026



Plan :

- 1. Géométrie hyperbolique**
- 2. Lua et LuaLaTeX**
- 3. Le package luahyperbolic**

Partie I

Introduction à la géométrie hyperbolique



hyperbolic geometry disk



Protection. Confidentialité. Tranquillité d'esprit.

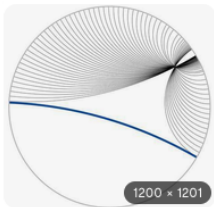


Tout **Images** Vidéos Actualités Plus

Duck.ai

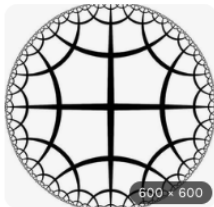


France Filtre parental : modérée Images générées par l'IA : afficher Date indifférente Toutes les tailles Toutes les couleurs Tous types Toutes les dispositions



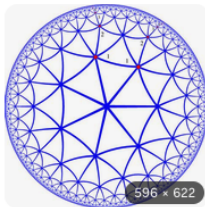
1200 x 1201

Poincaré disk model - Wikipedia
W en.wikipedia.org



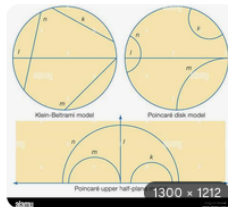
600 x 600

Conformal Models of Hyperbolic Geometry
bulatov.org



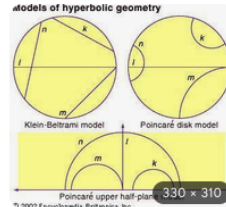
596 x 622

The hyperbolic disk - a seven-pointed star
R⁶ researchgate.net



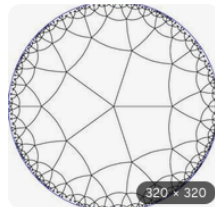
1300 x 1212

Three models of hyperbolic geometry
alamy.com



330 x 310

Poincaré disk model | geometry | Britannica
britannica.com



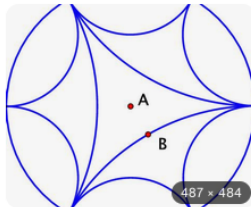
320 x 320

A tessellation of the Poincaré disk model
R⁶ researchgate.net



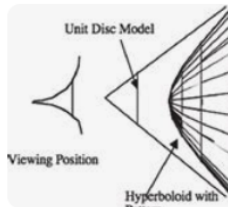
674 x 326

Geometries of Surfaces
pi.math.cornell.edu



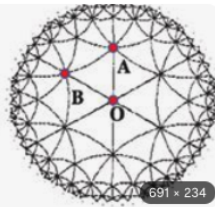
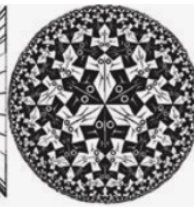
487 x 484

A tiling of the hyperbolic plane, in the Poincaré disk model
R⁶ researchgate.net



15 A genesis of the Poincaré disk for hyperbolic geometry

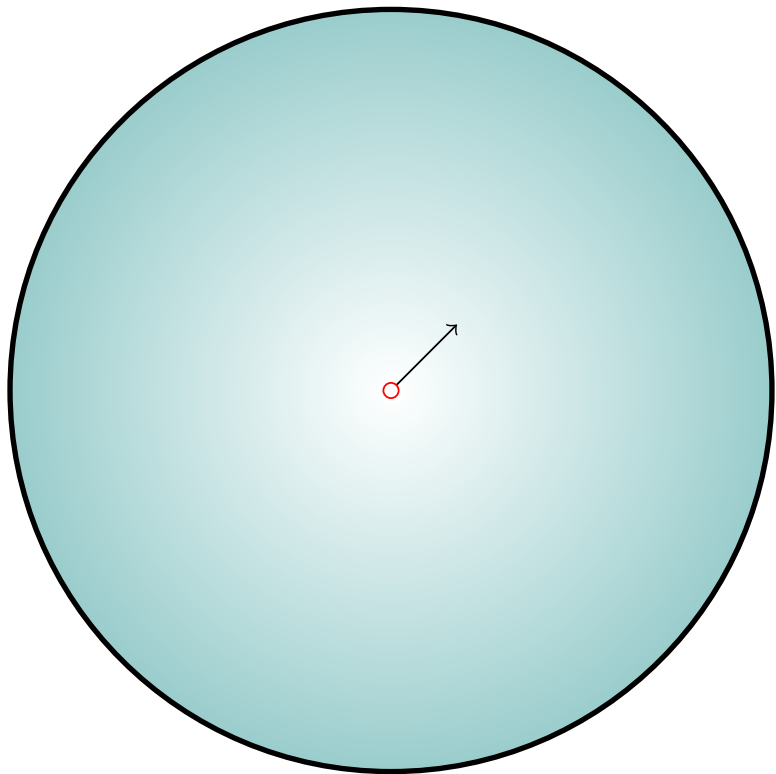
15 A genesis of the Poincaré disk for hyperbolic geometry | Download
R⁶ researchgate.net

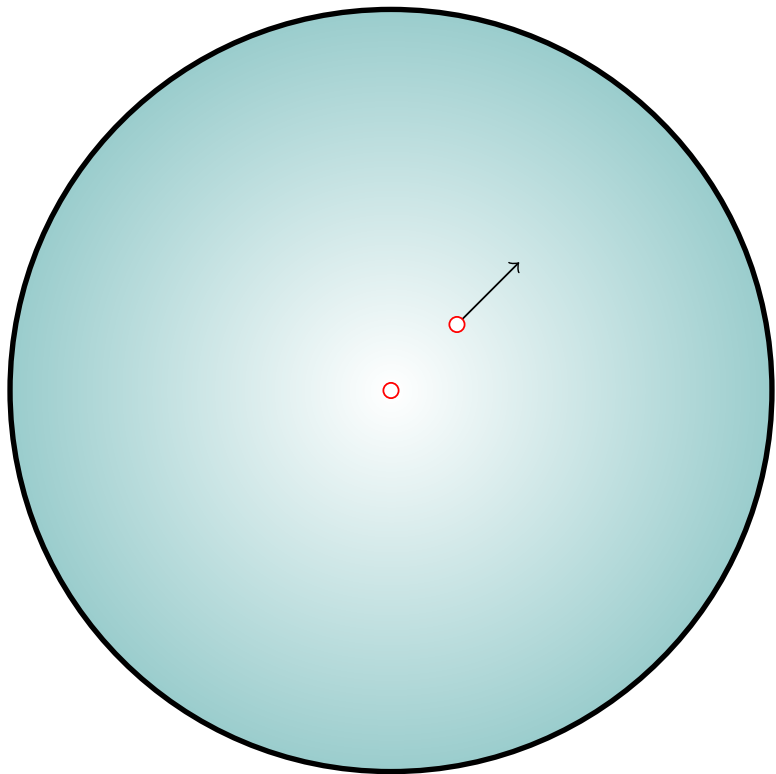


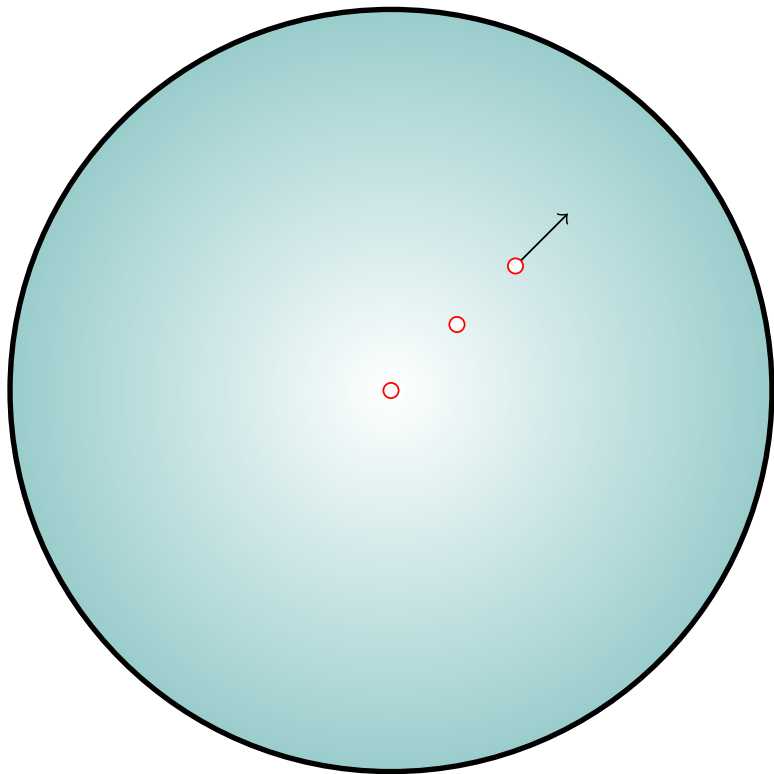
691 x 234

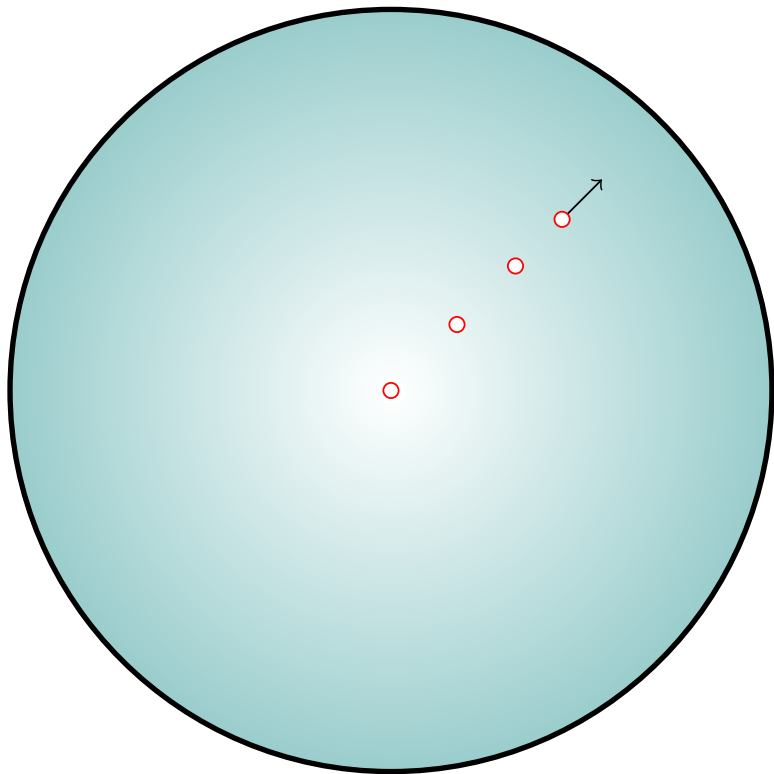
Partagez vos commentaires

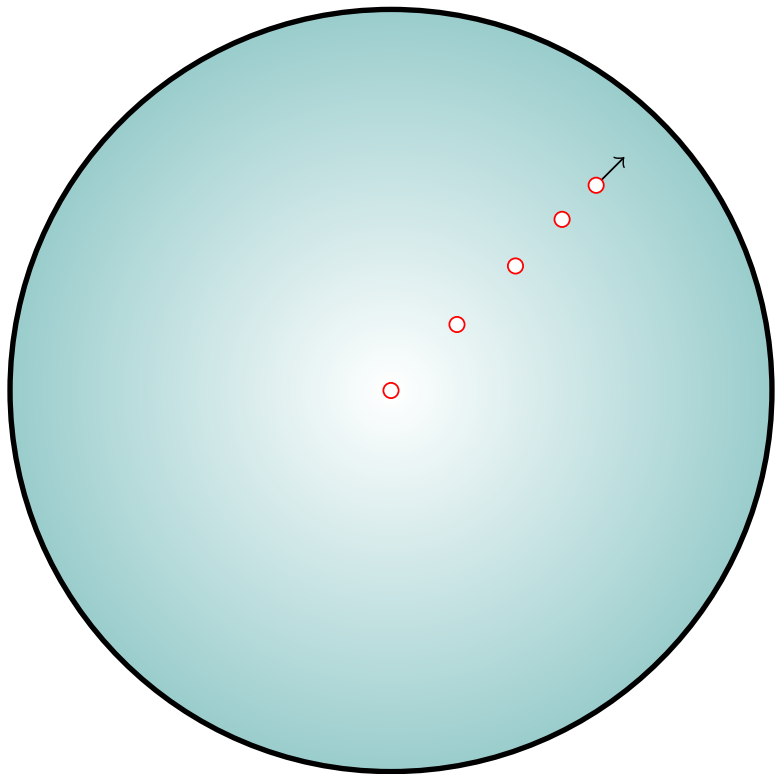
Le modèle du disque de Poincaré

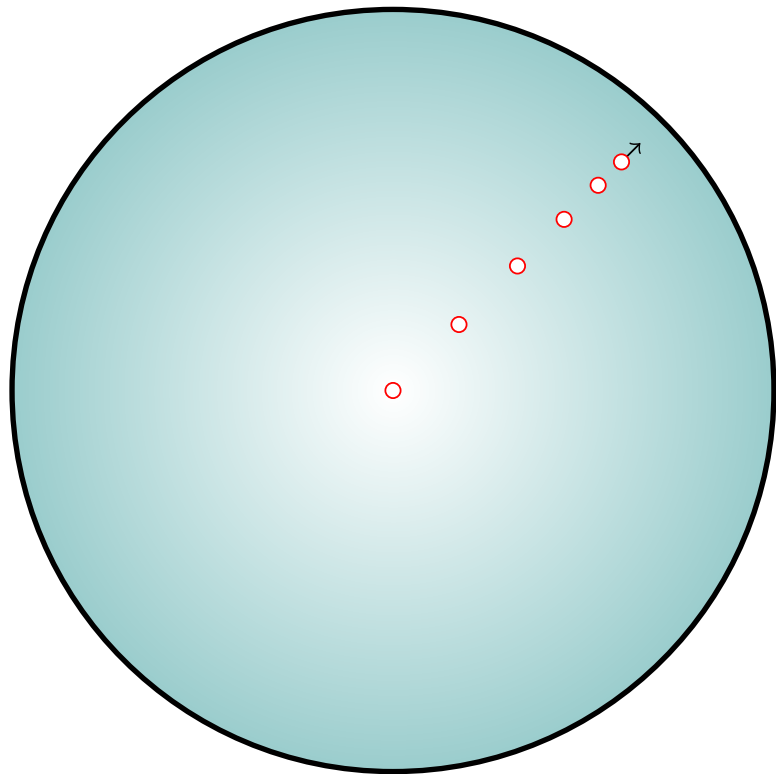


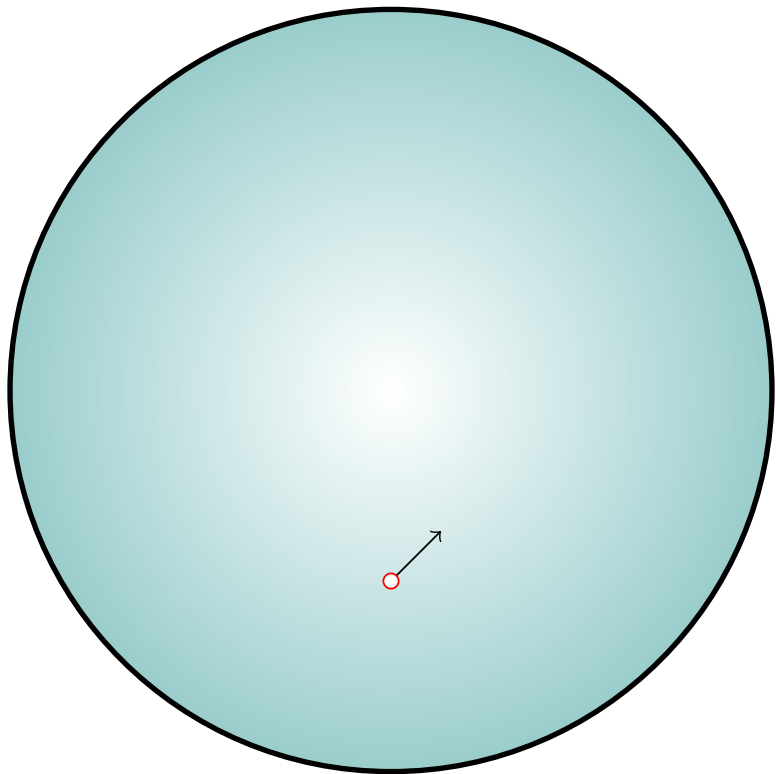


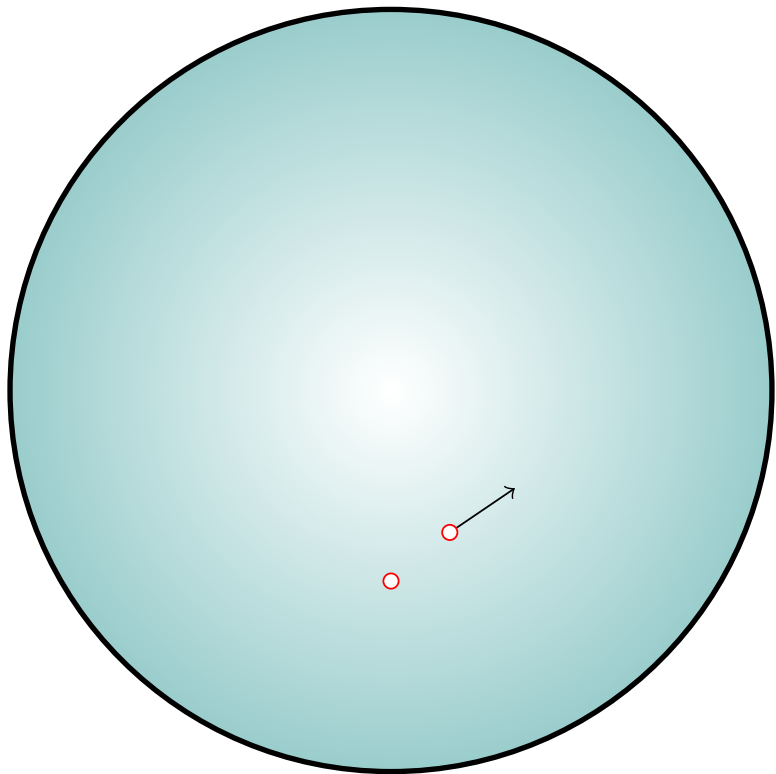


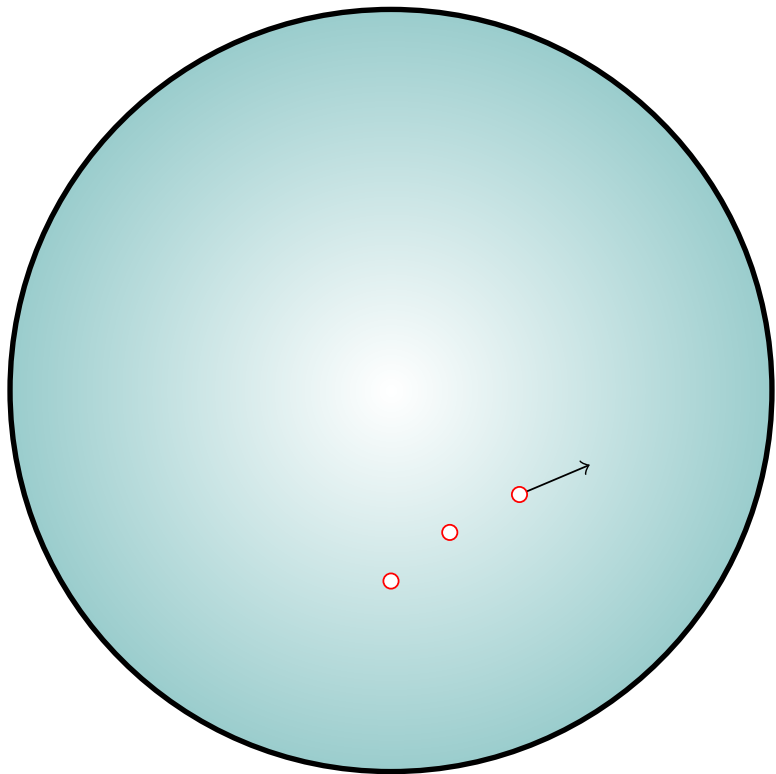


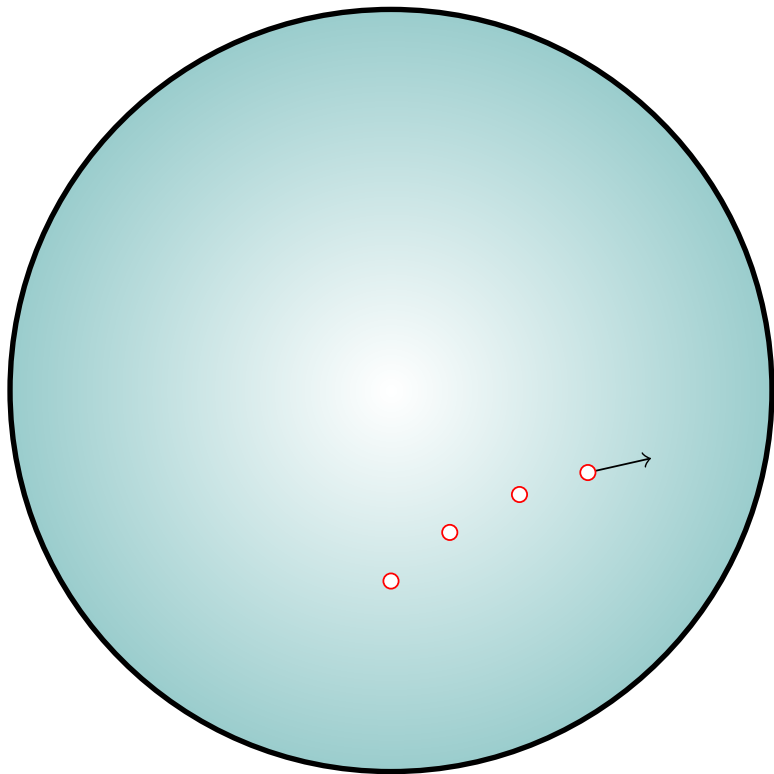


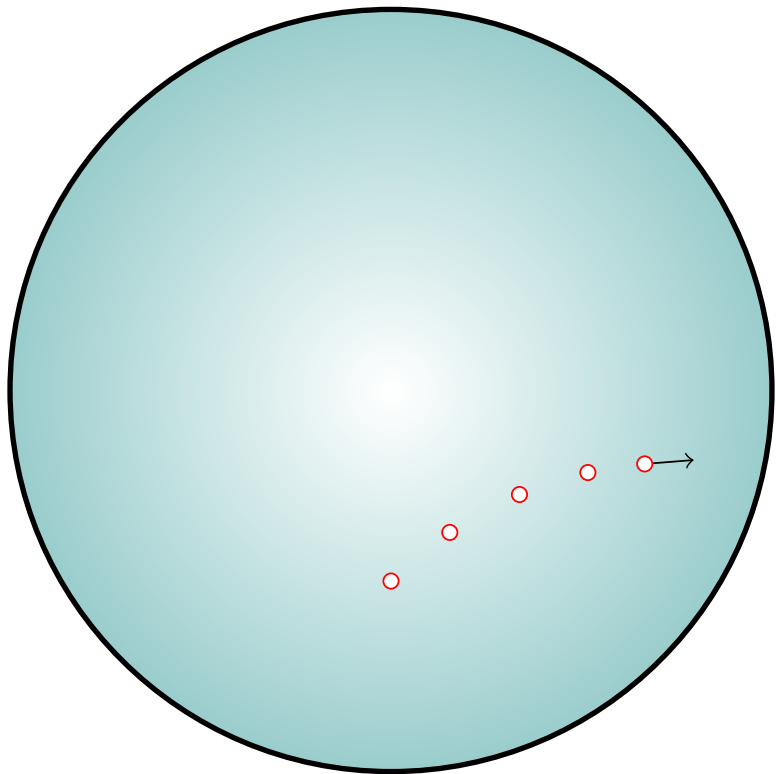


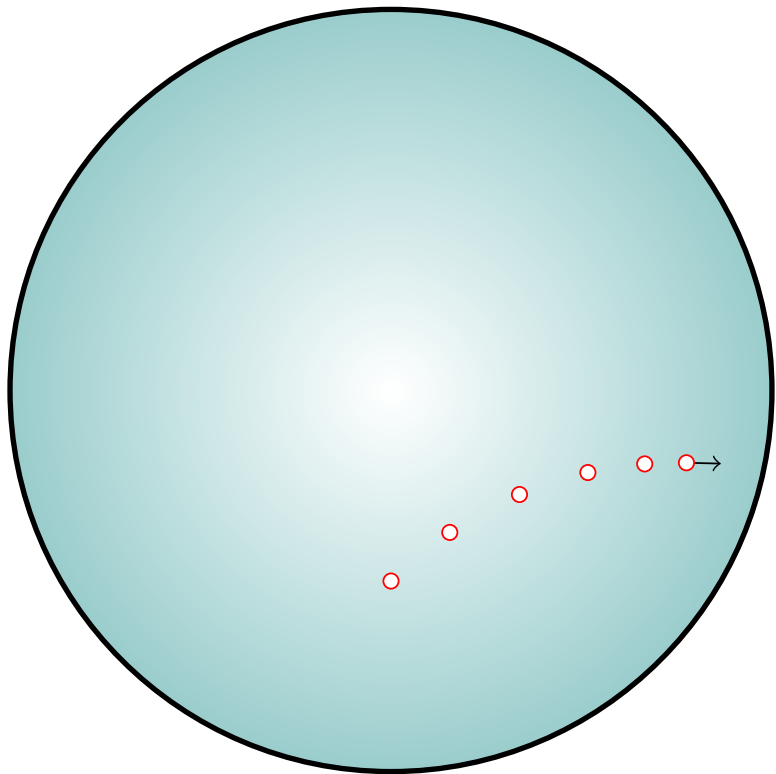




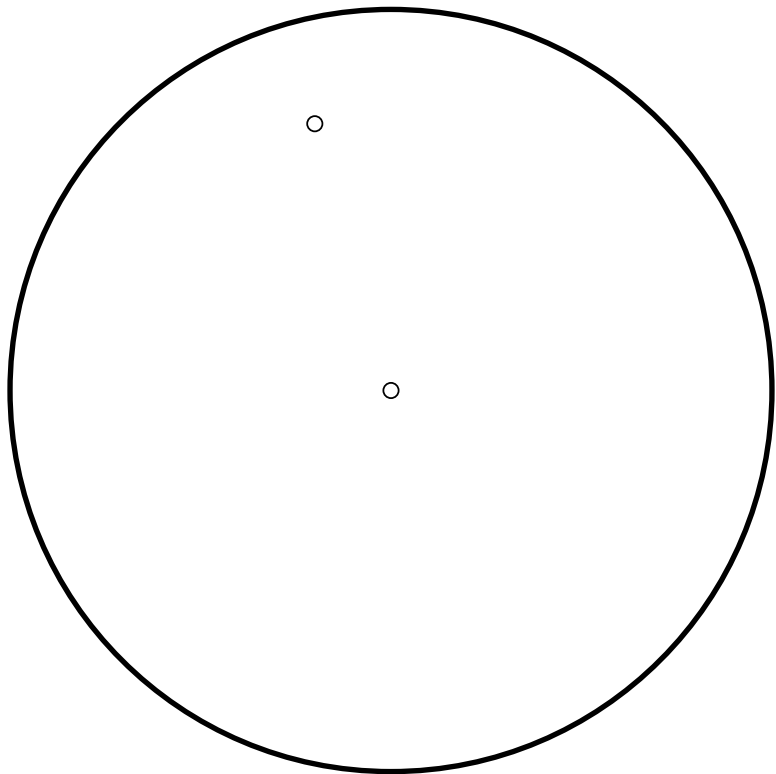


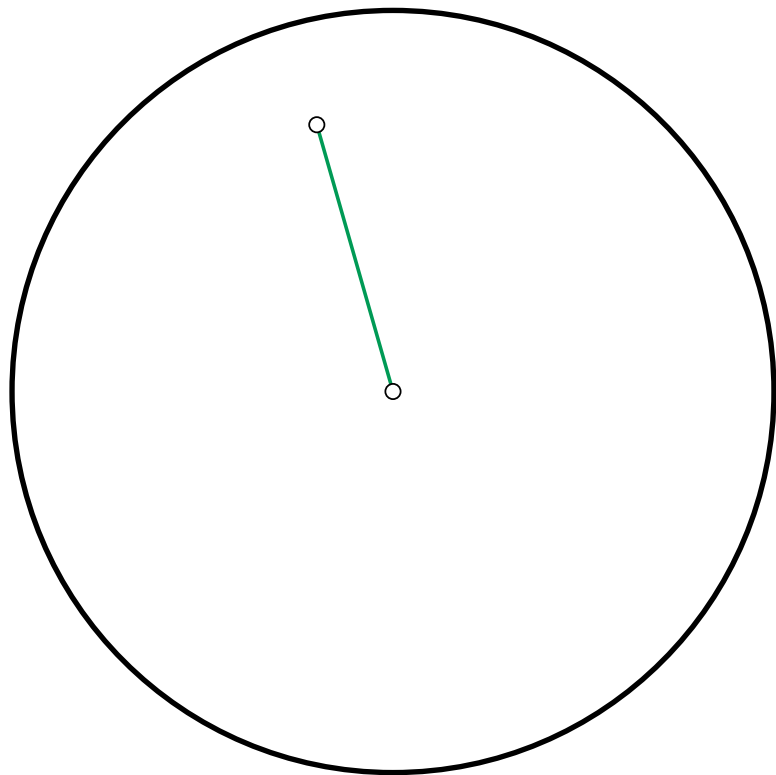


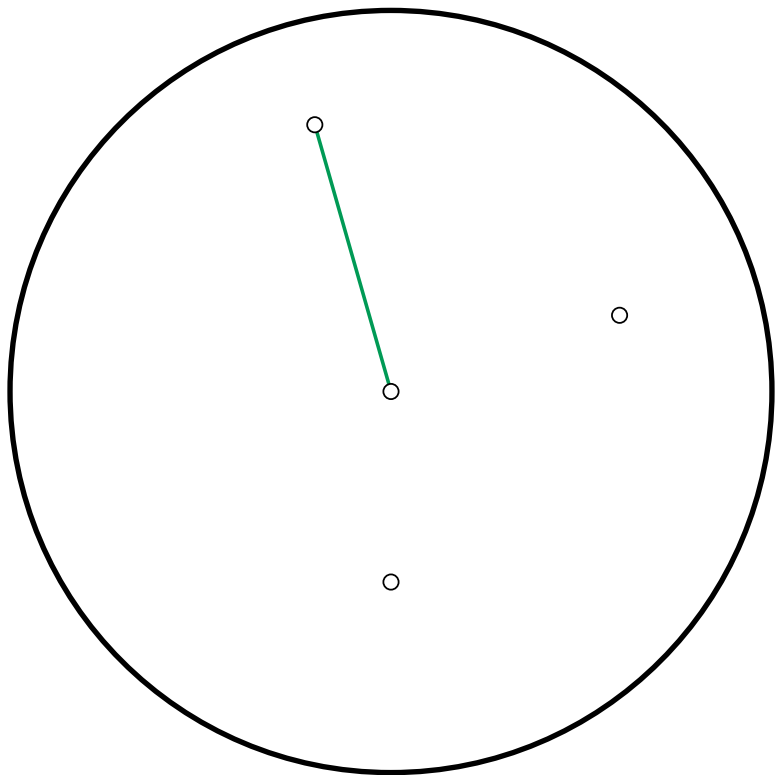


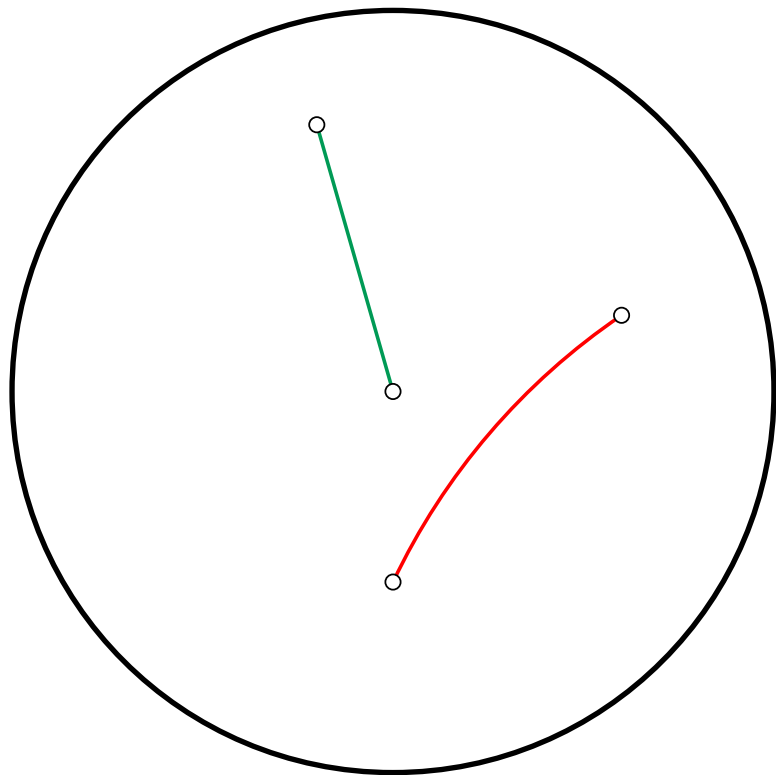


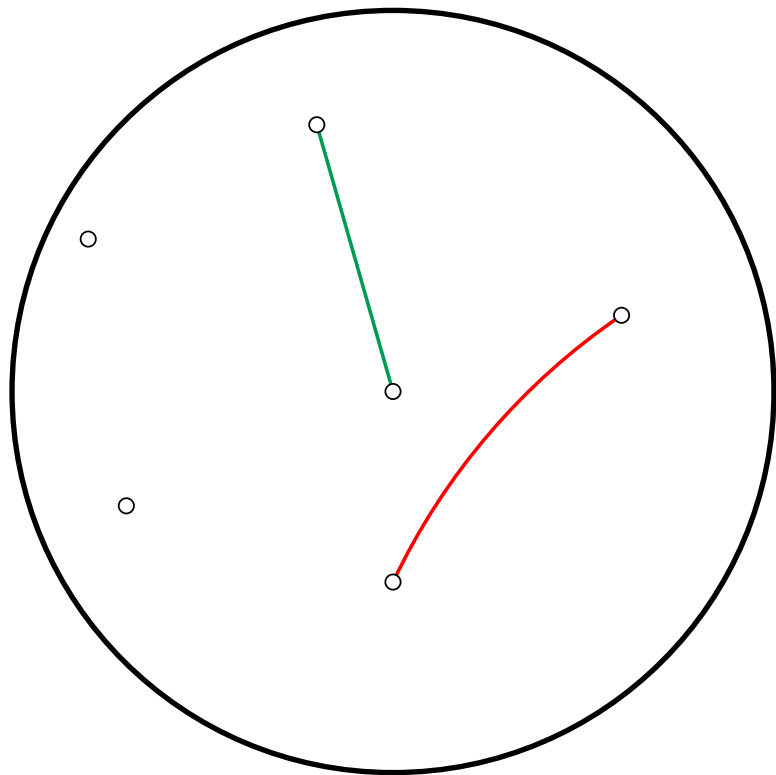
Géodésiques, segments et droites

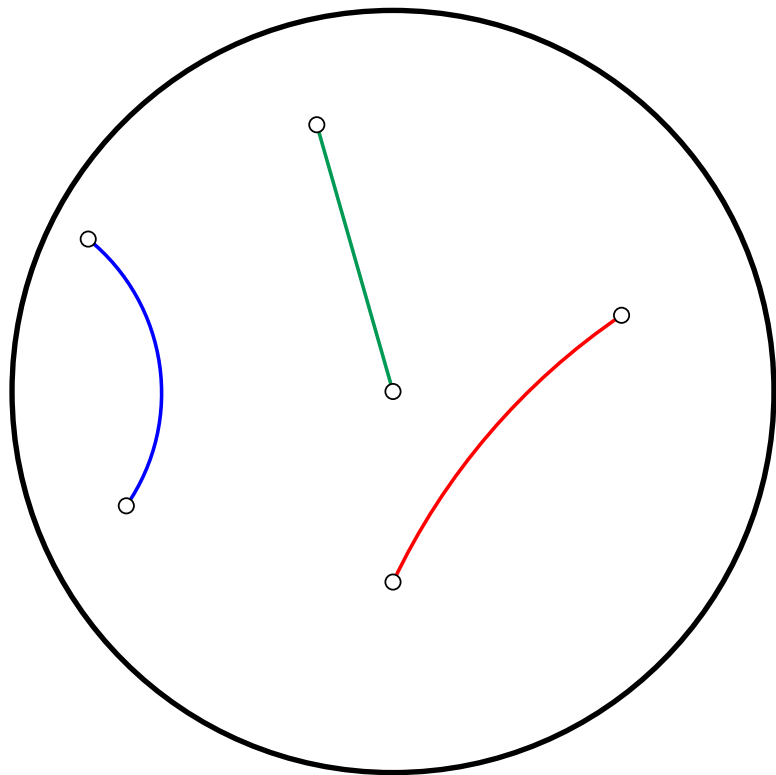


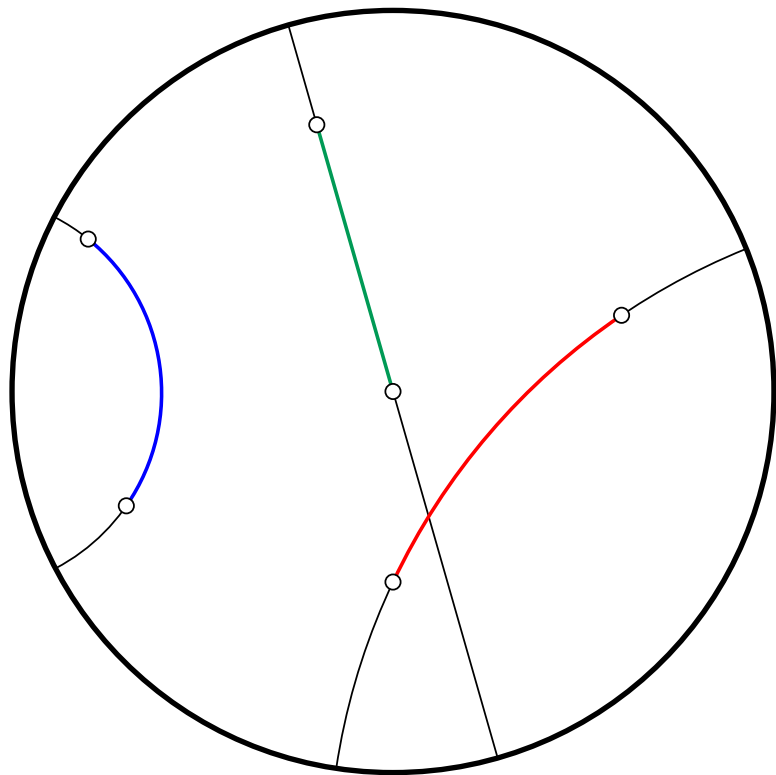




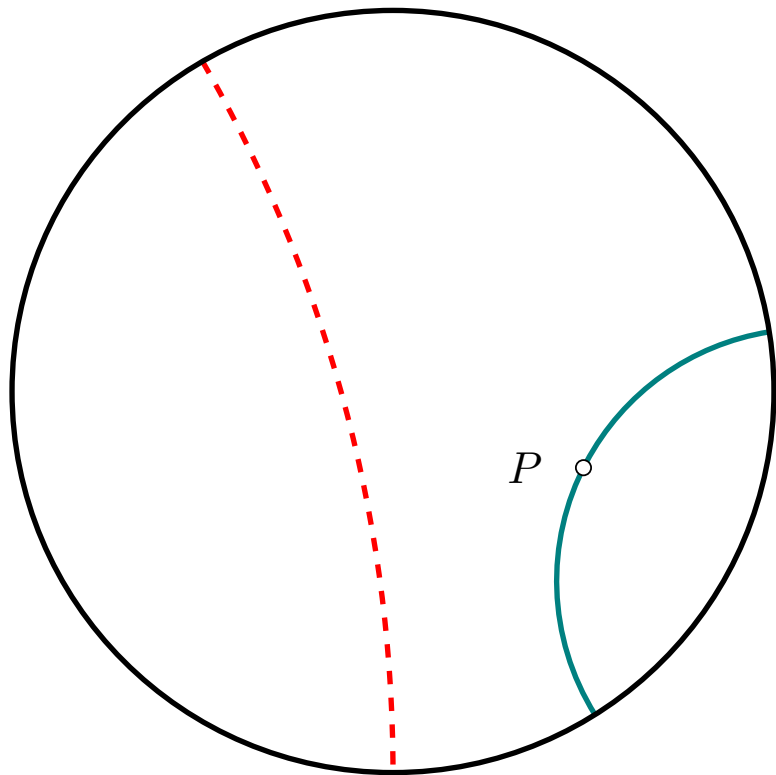


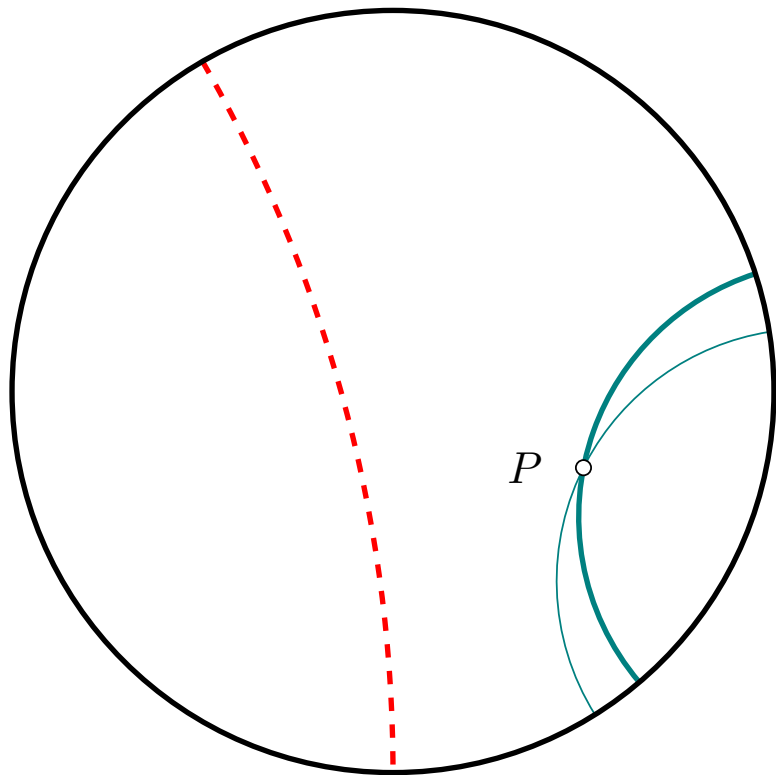


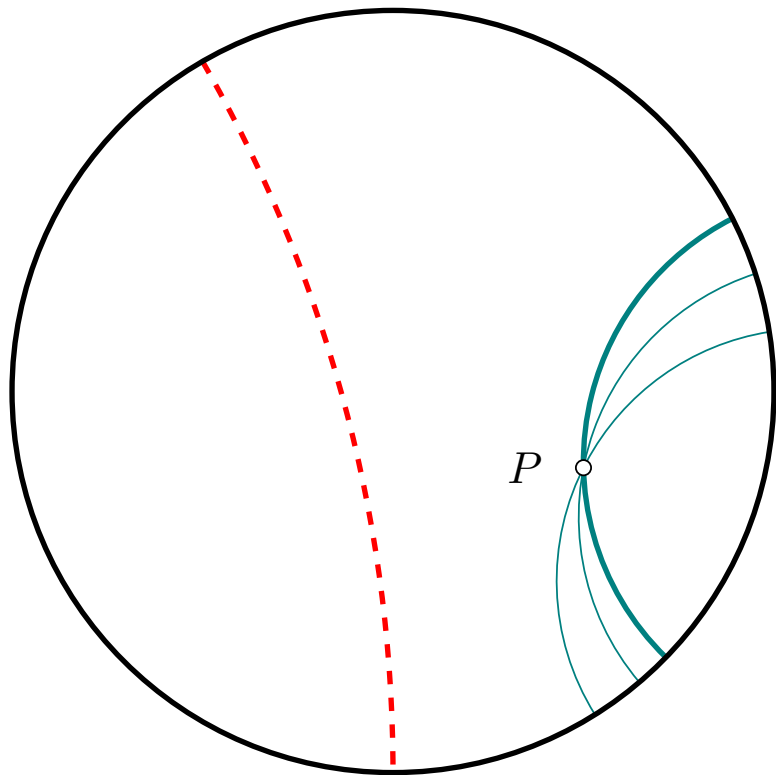


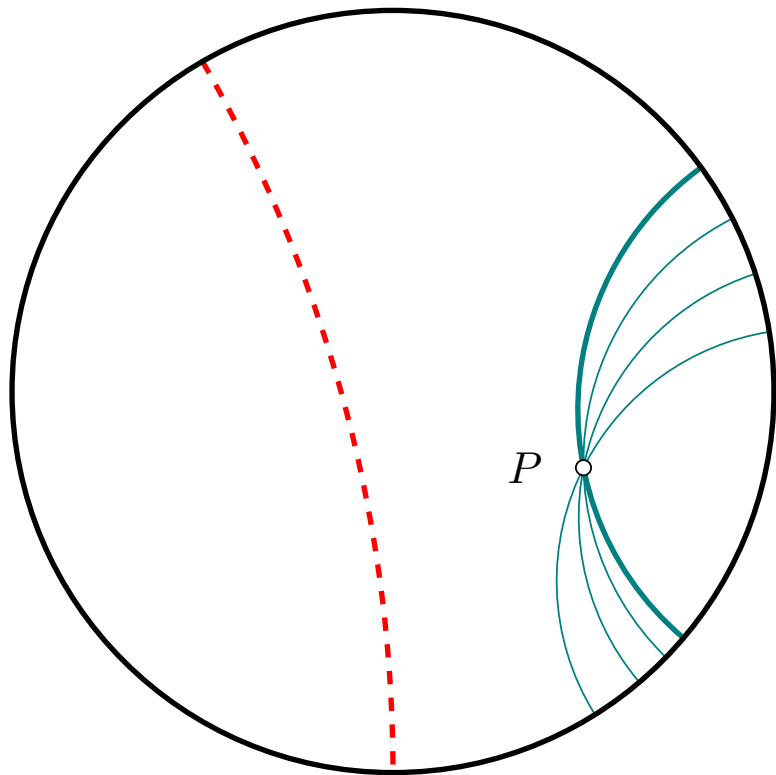


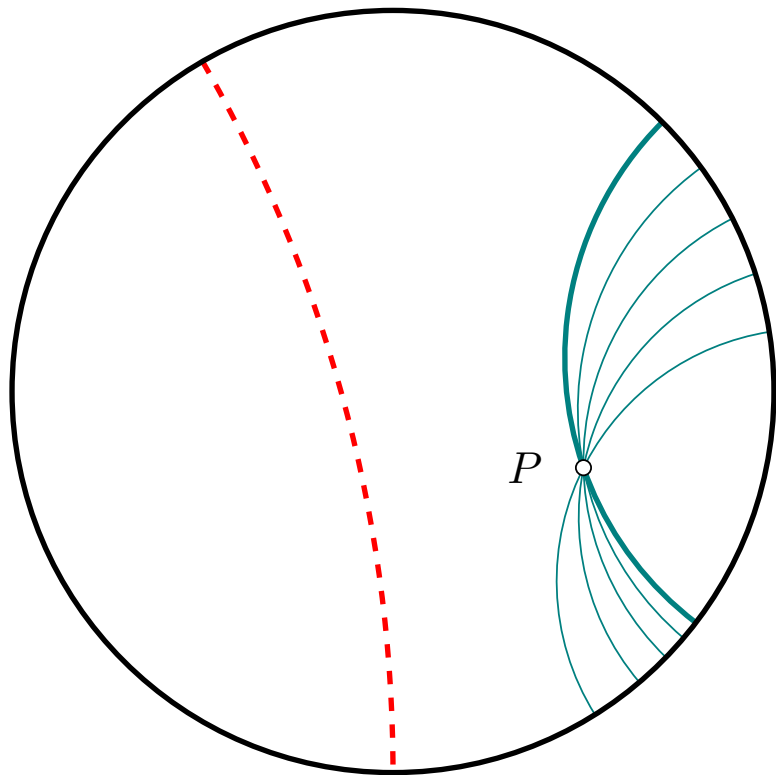
Attention !!
Le 5ème postulat d'Euclide
tombe en défaut !



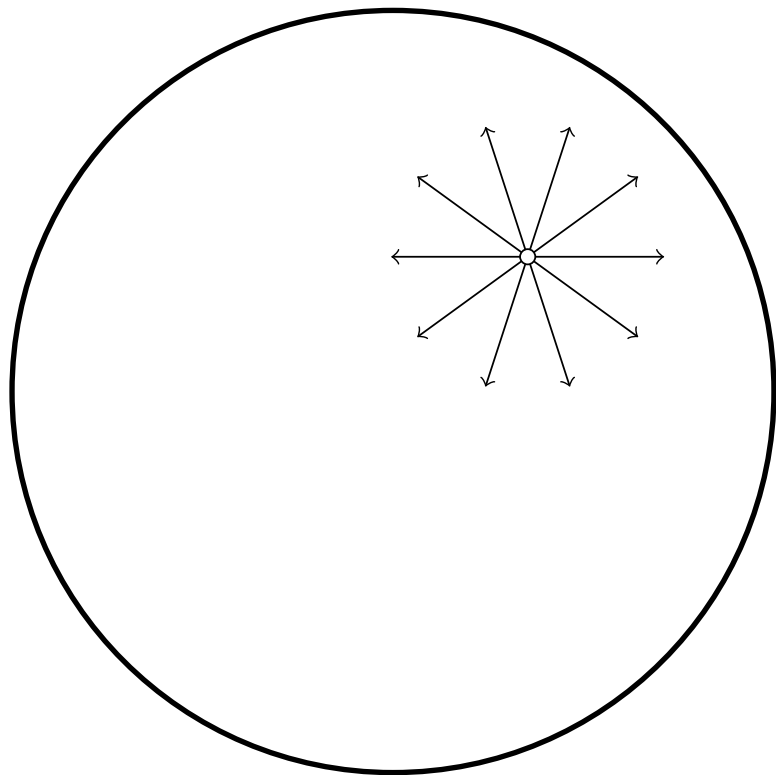


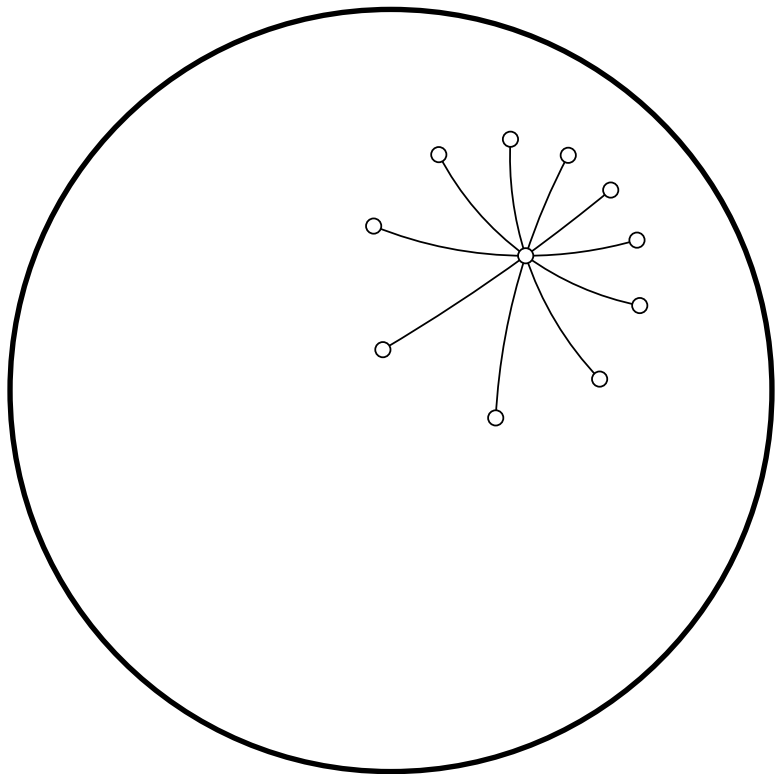


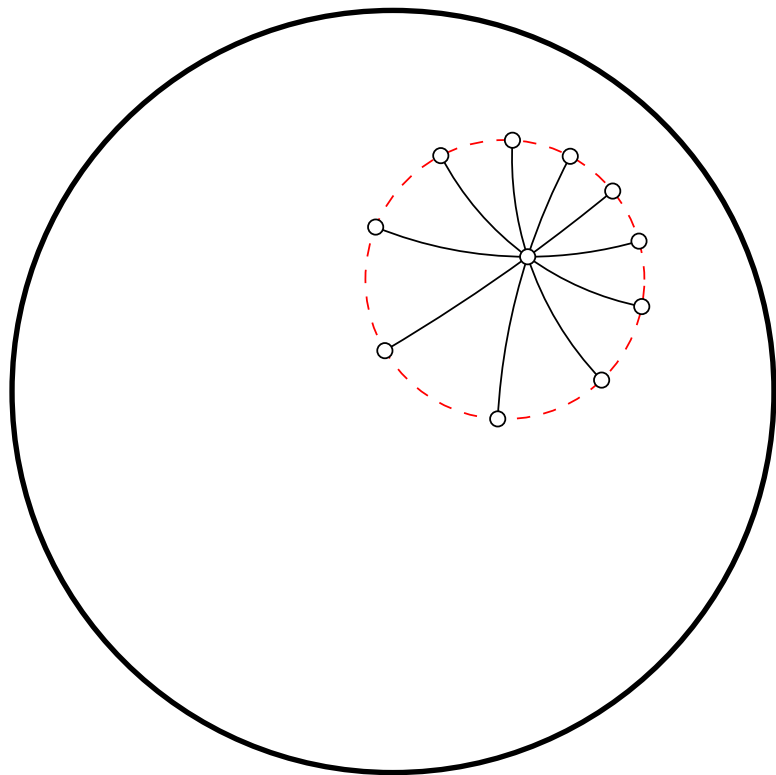


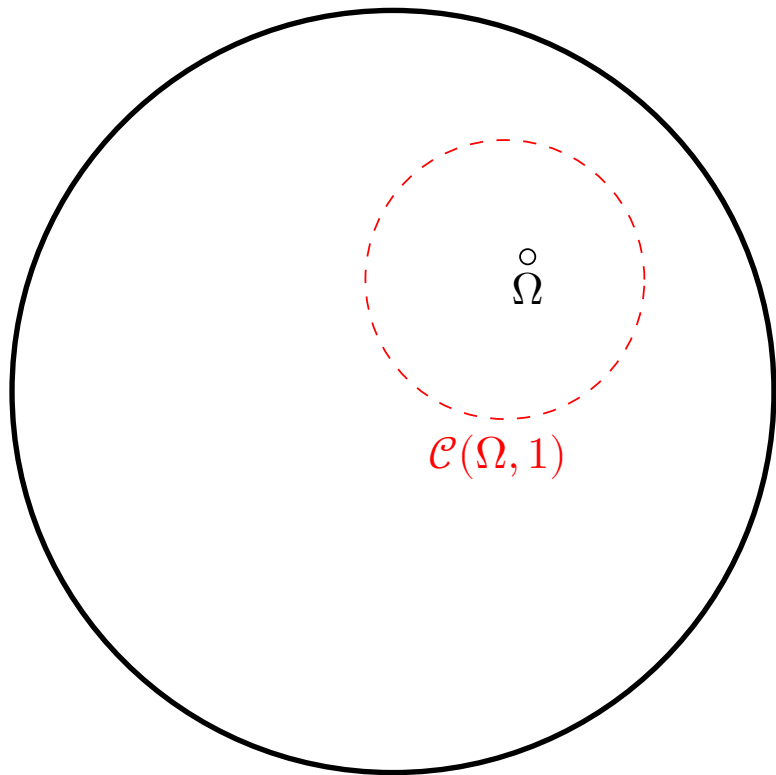


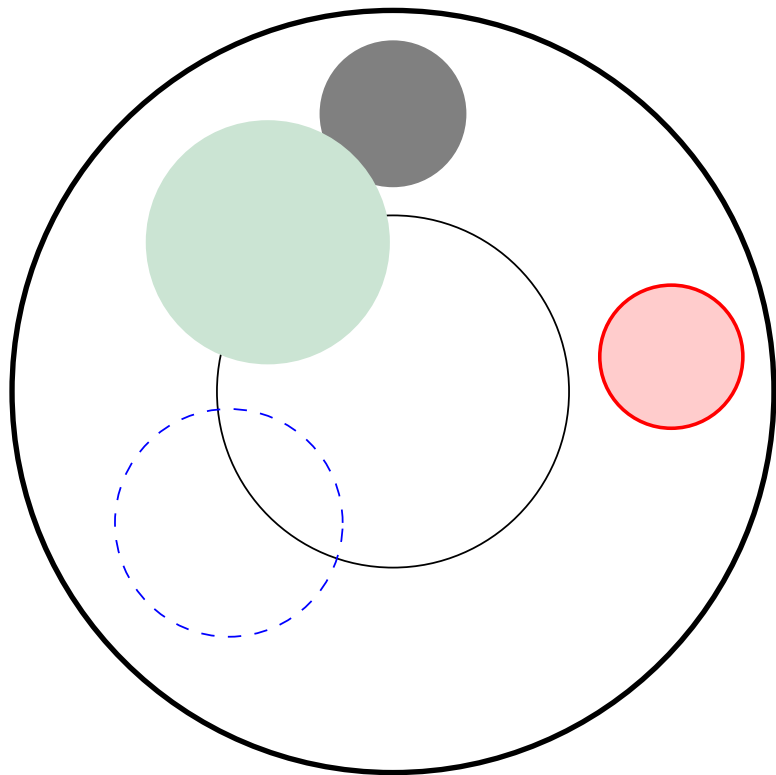
Cercles





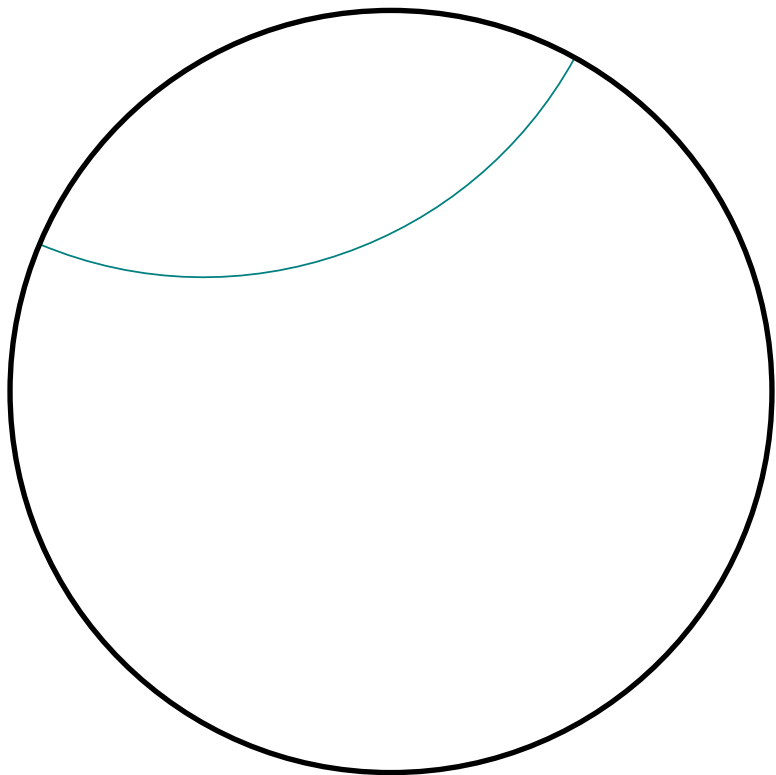


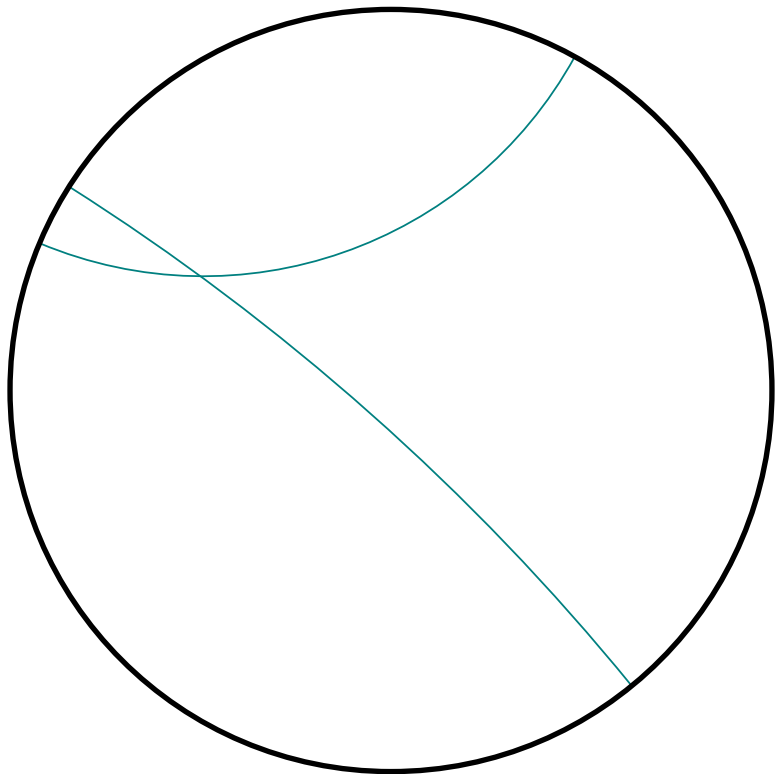


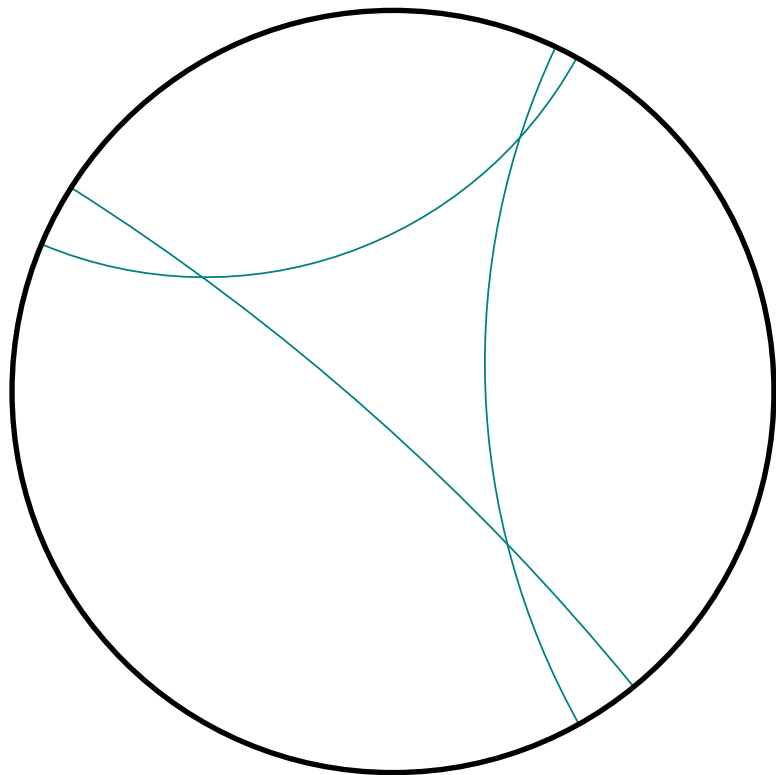


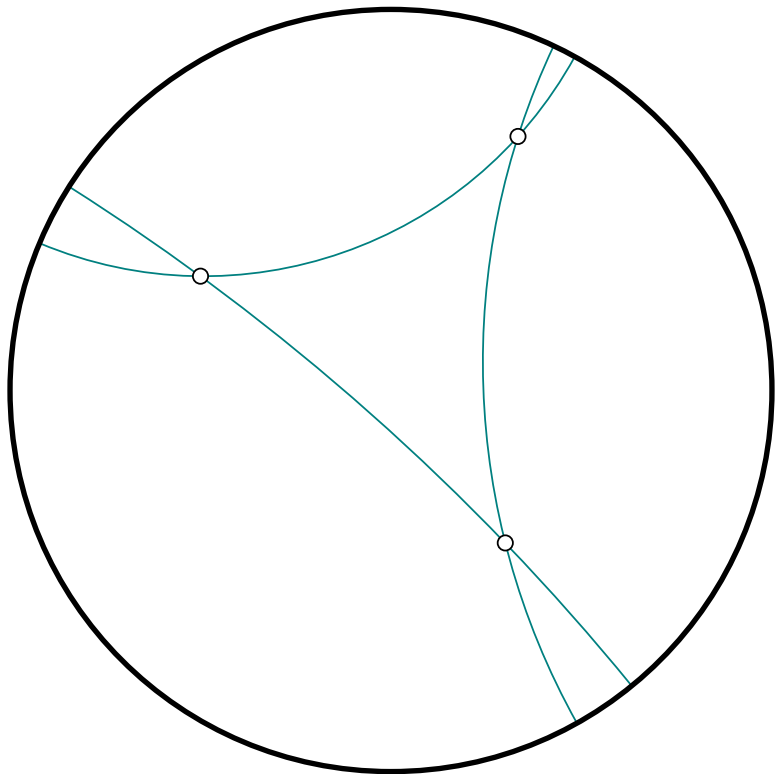
Triangles :

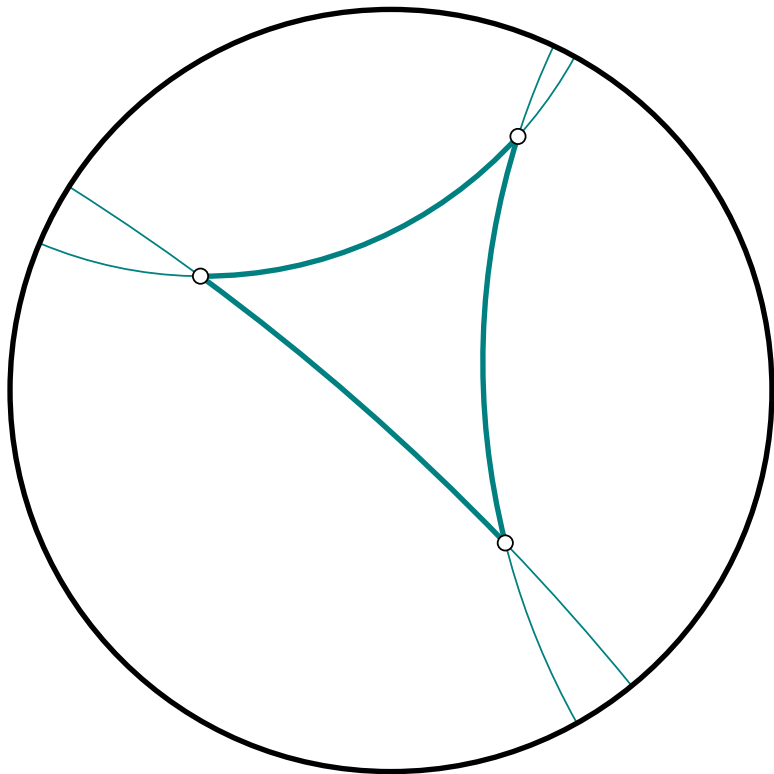
Du normal et du moins normal

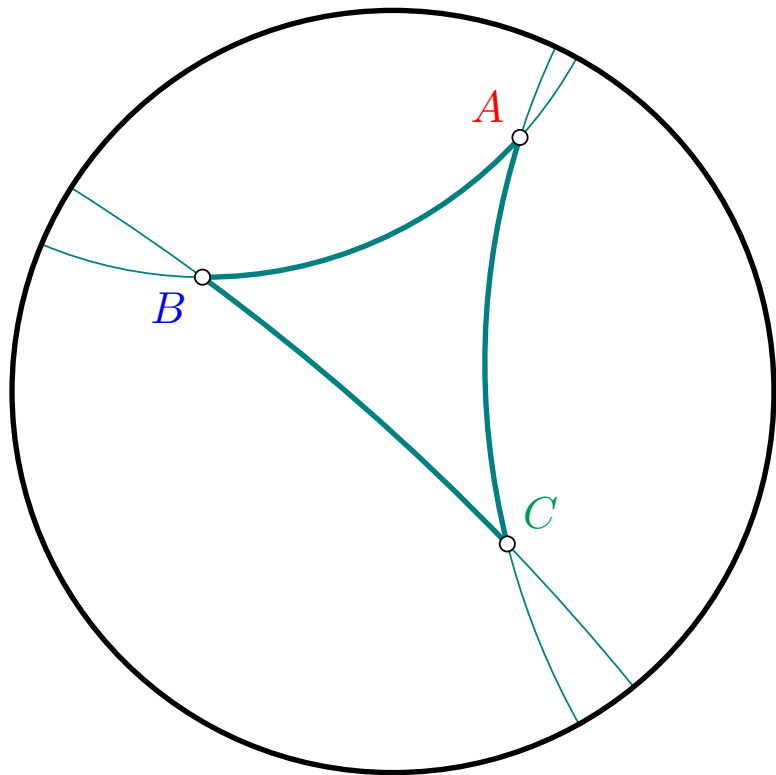


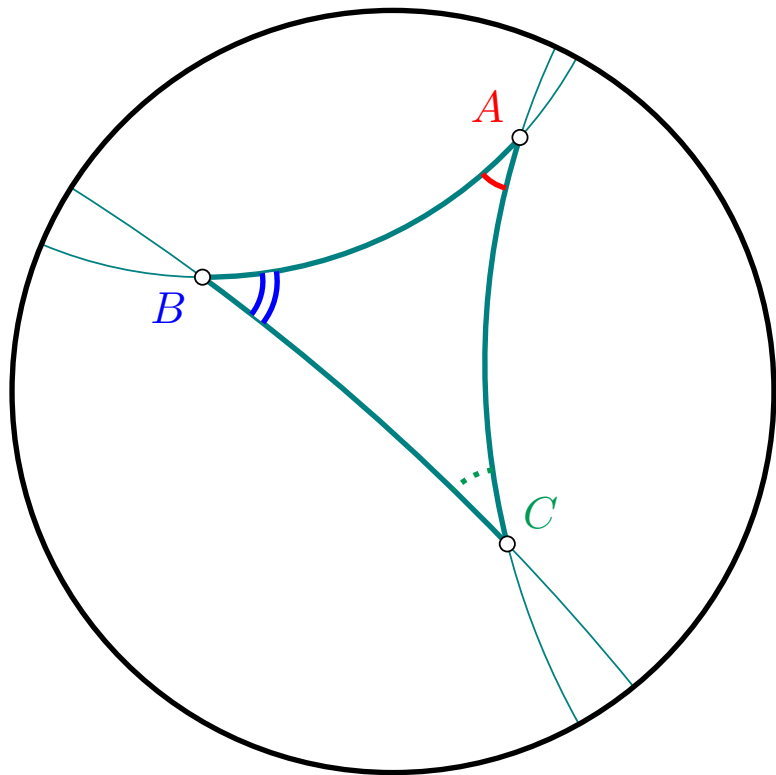


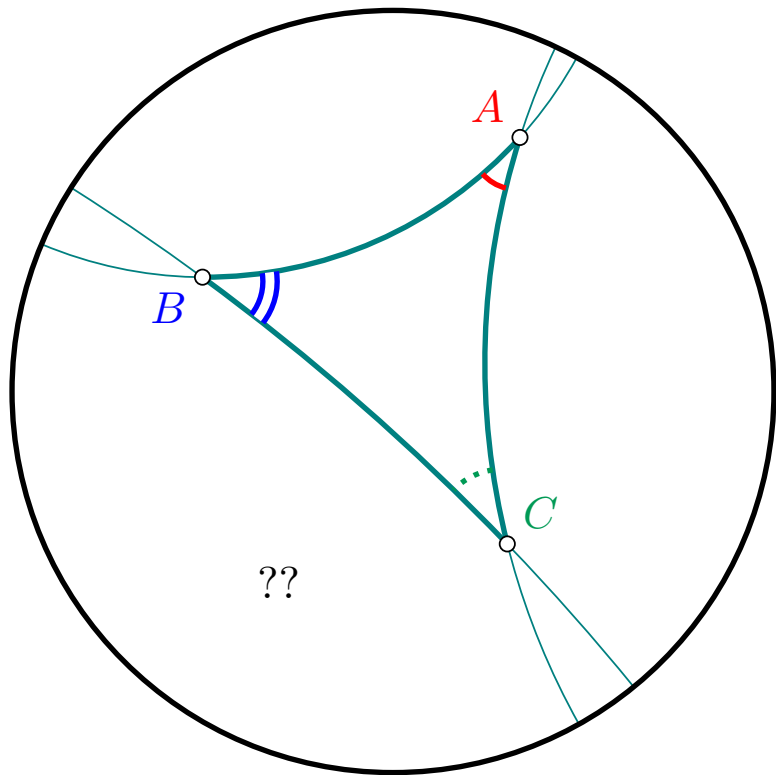


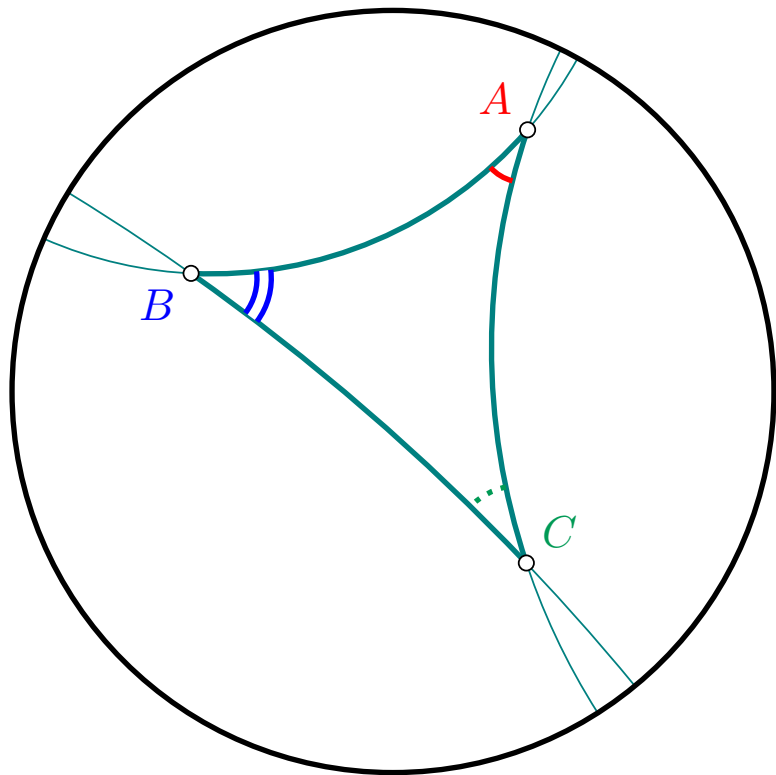


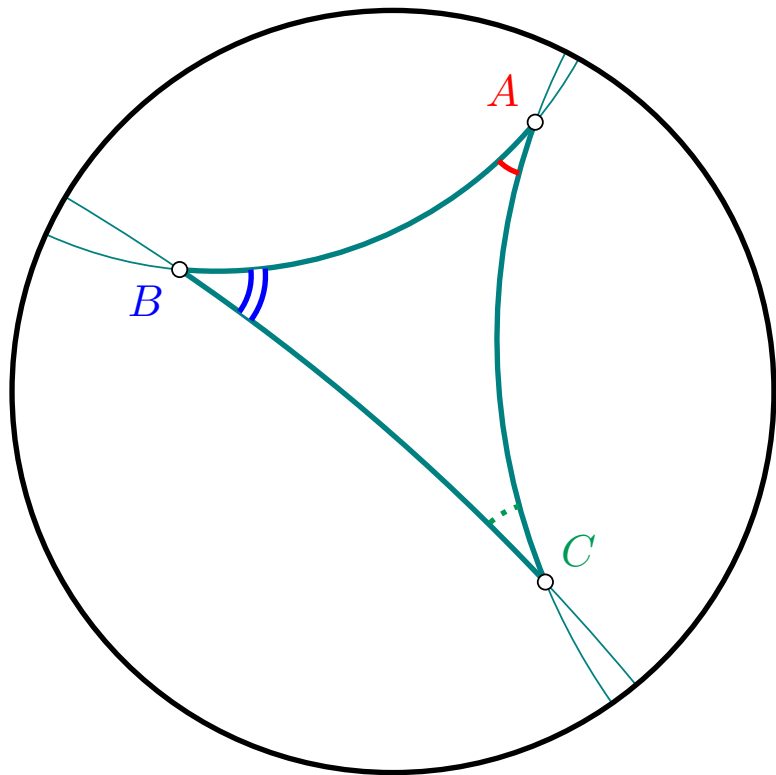


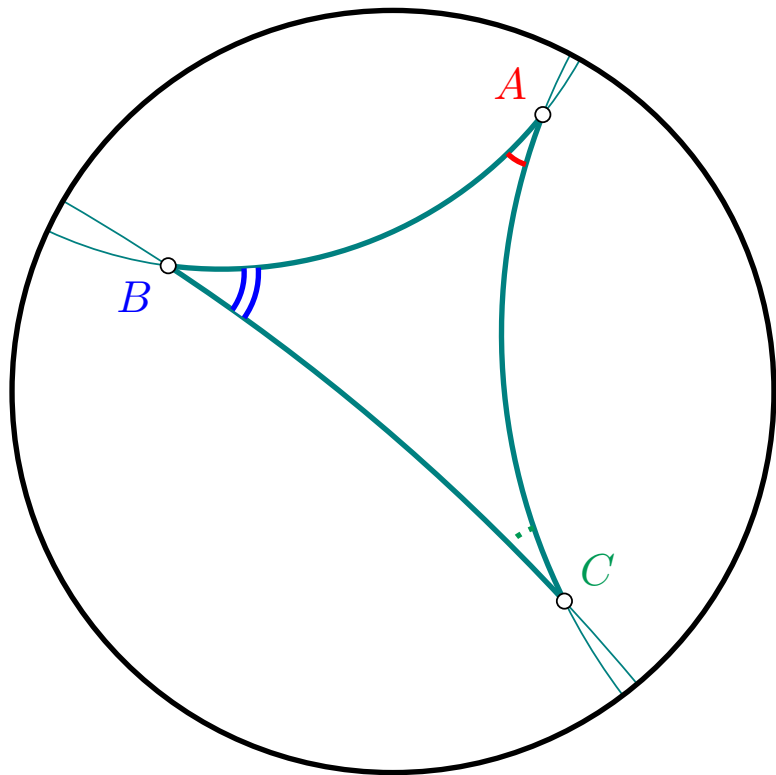


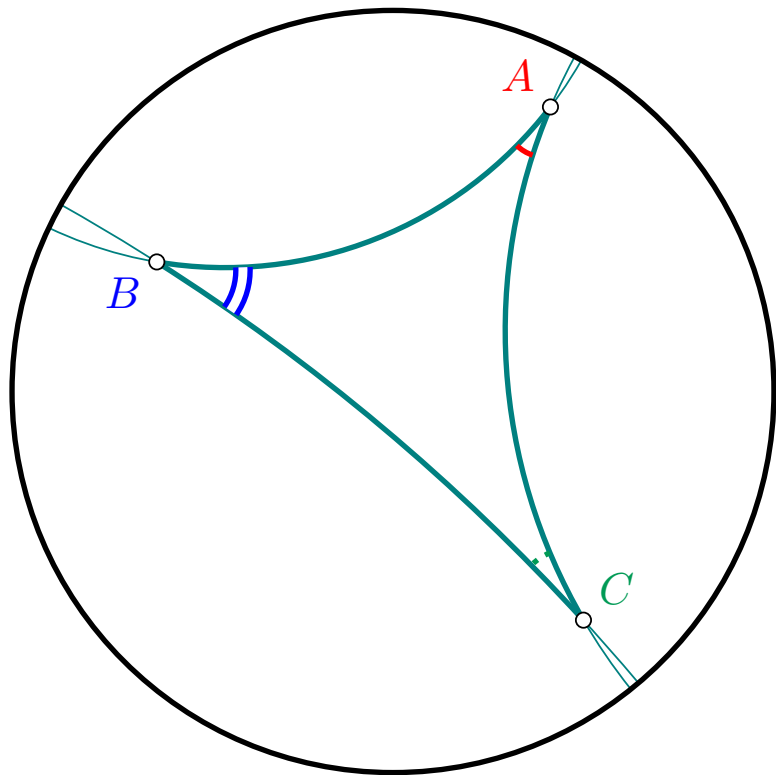


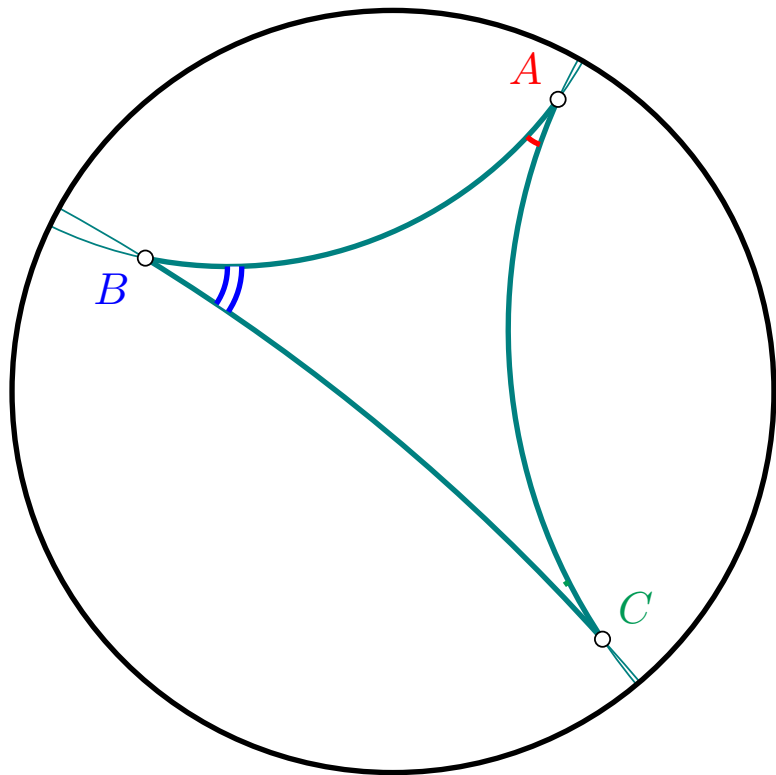


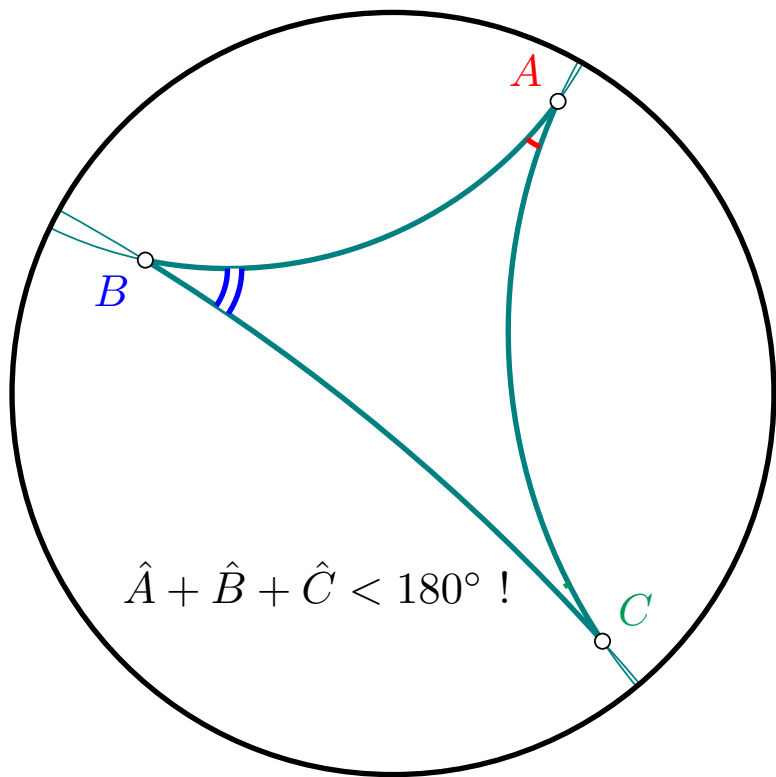




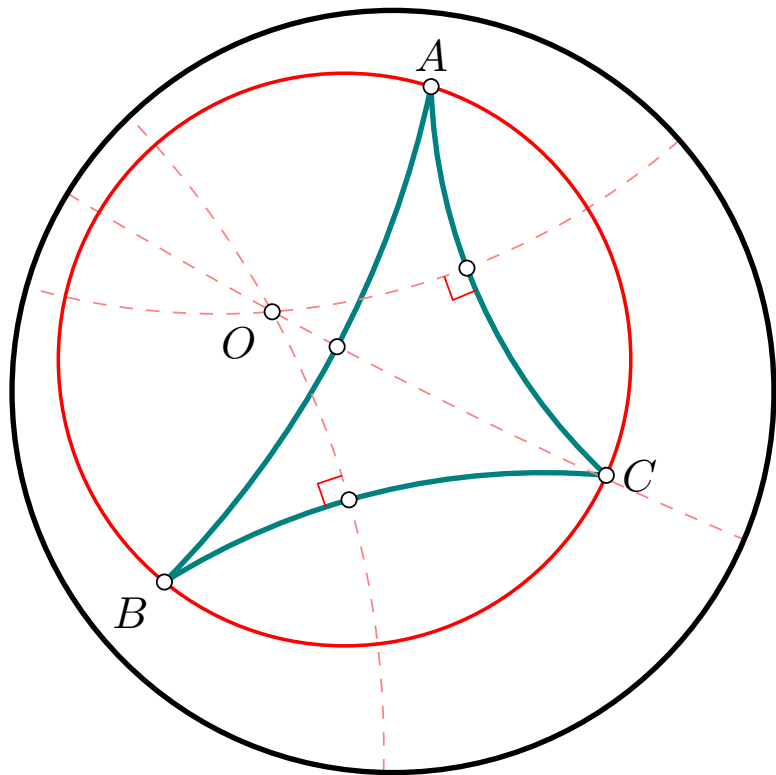


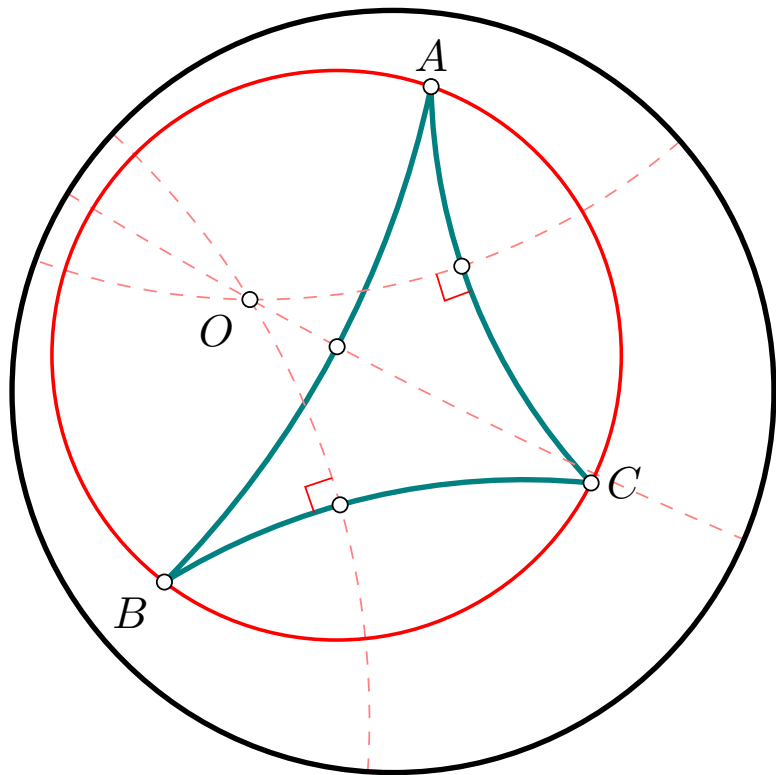


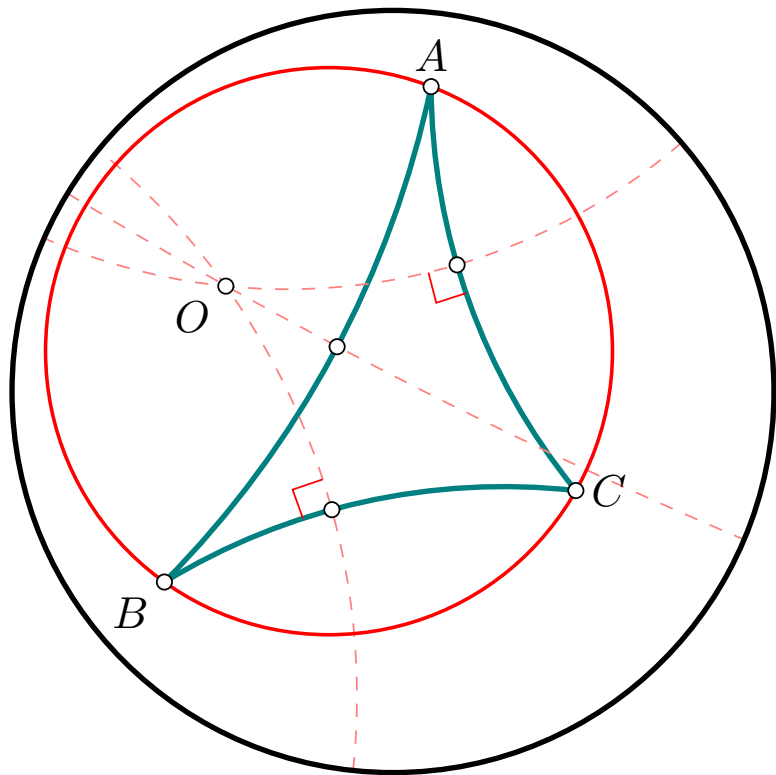


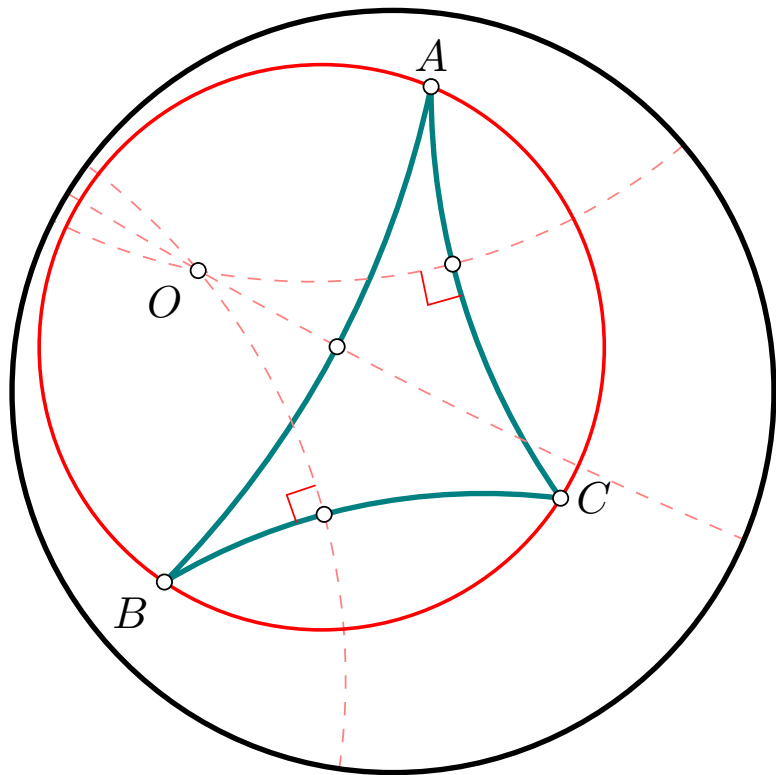


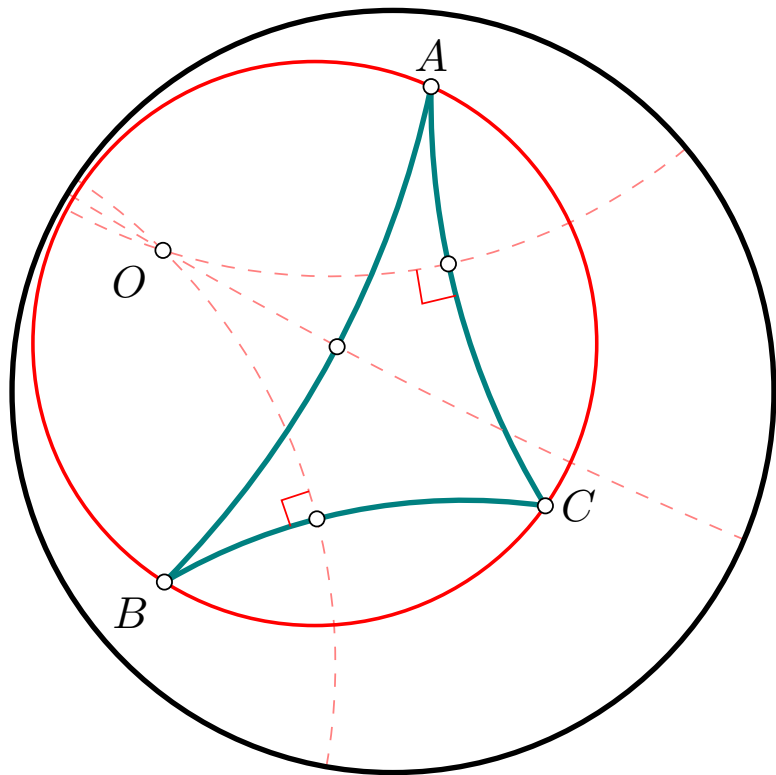
Triangles :
Autres bizarreries
(Cercles circonscrits)

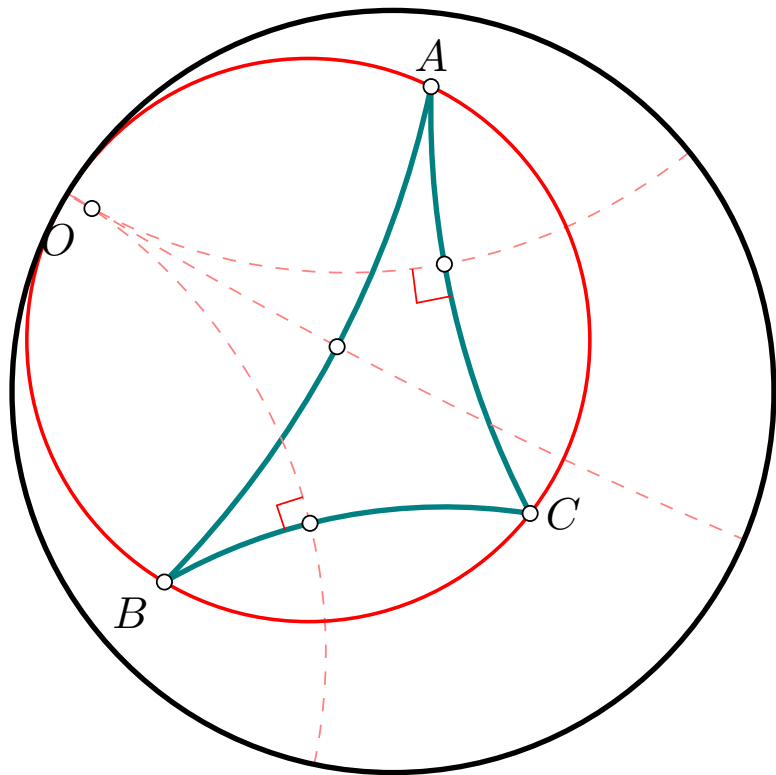


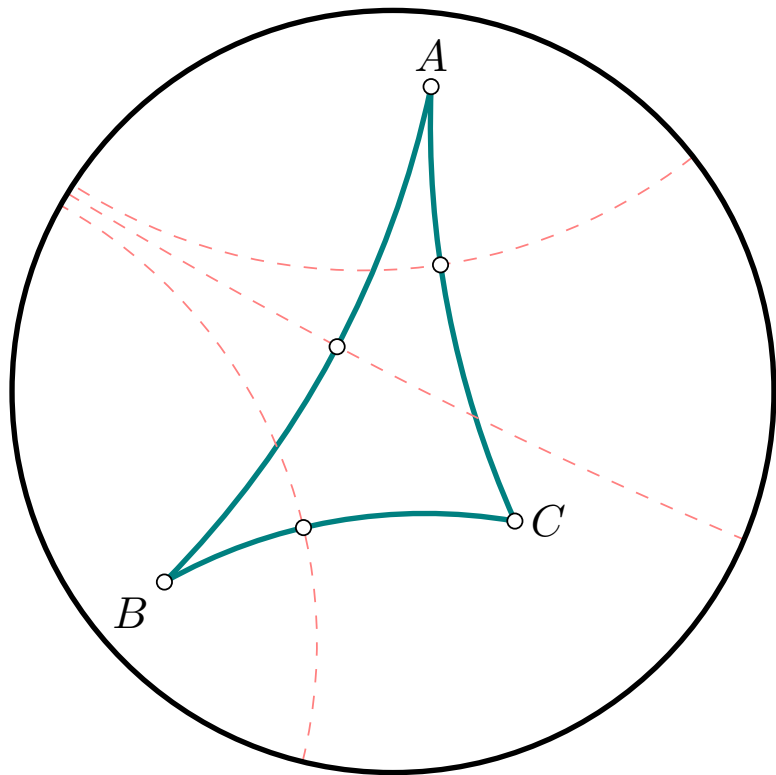






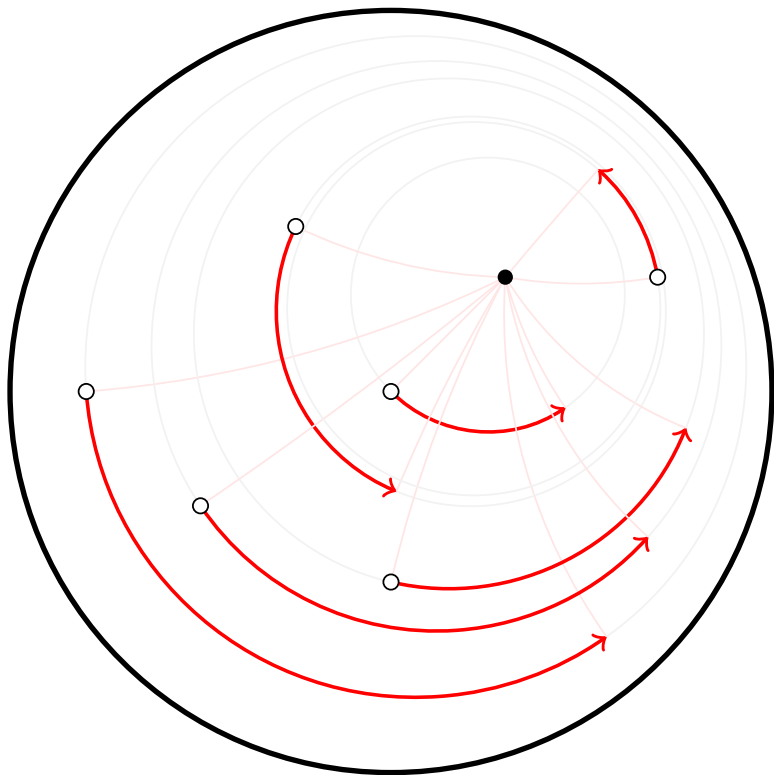




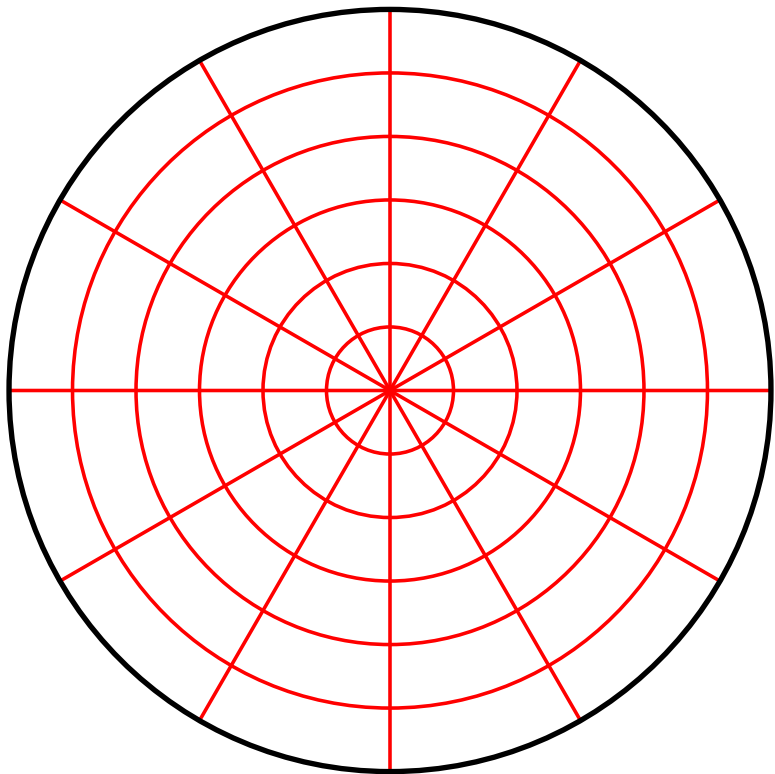


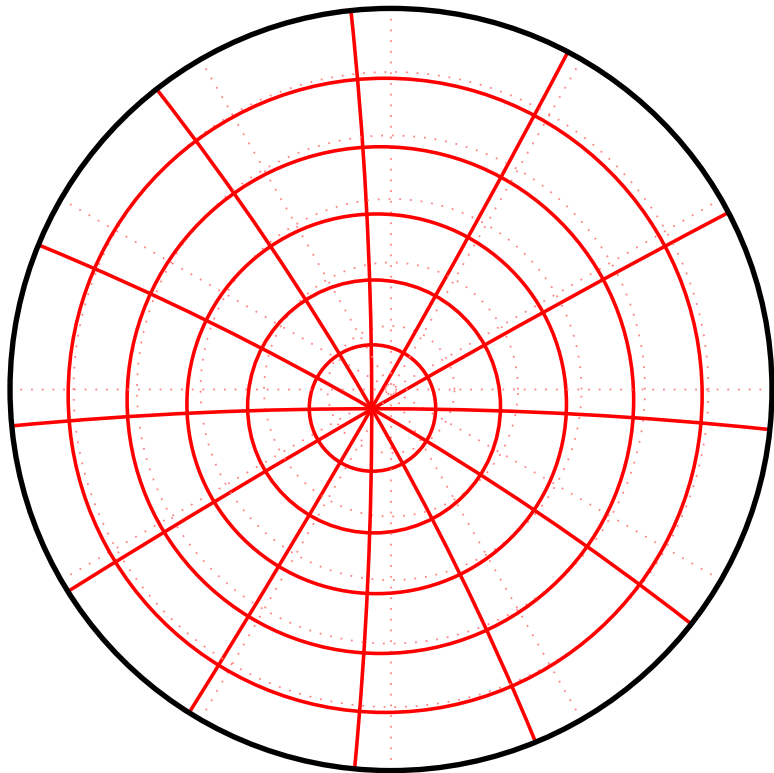
Transformations (isométries) du plan hyperbolique

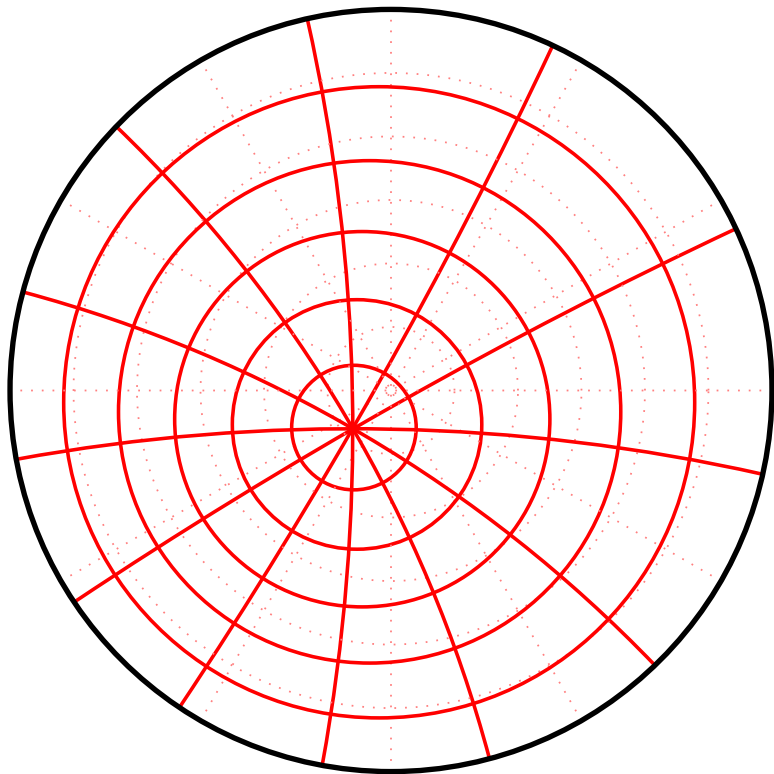
Automorphismes elliptiques (Analogue des rotations)

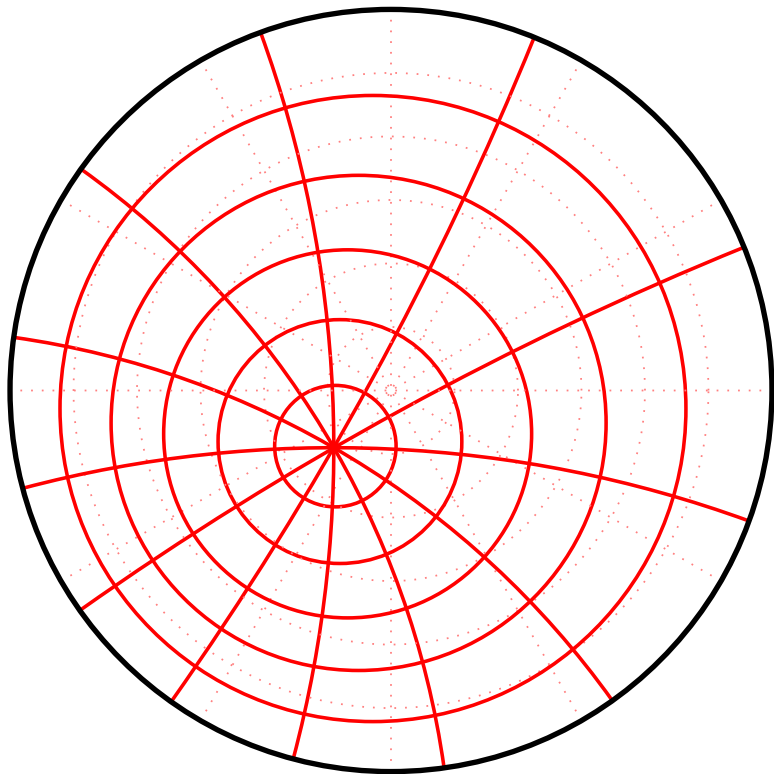


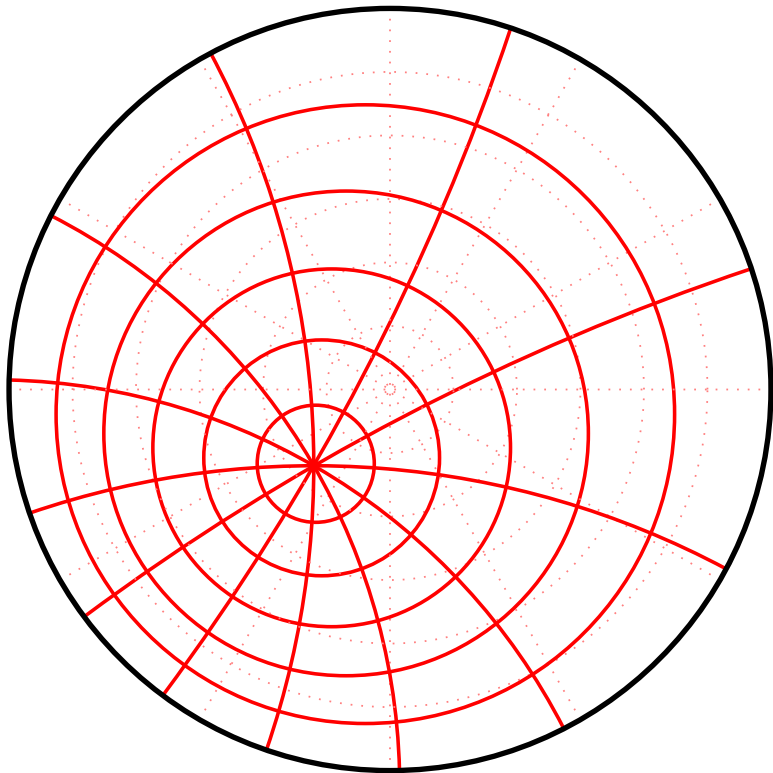
Automorphismes hyperboliques (Analogue des translations)

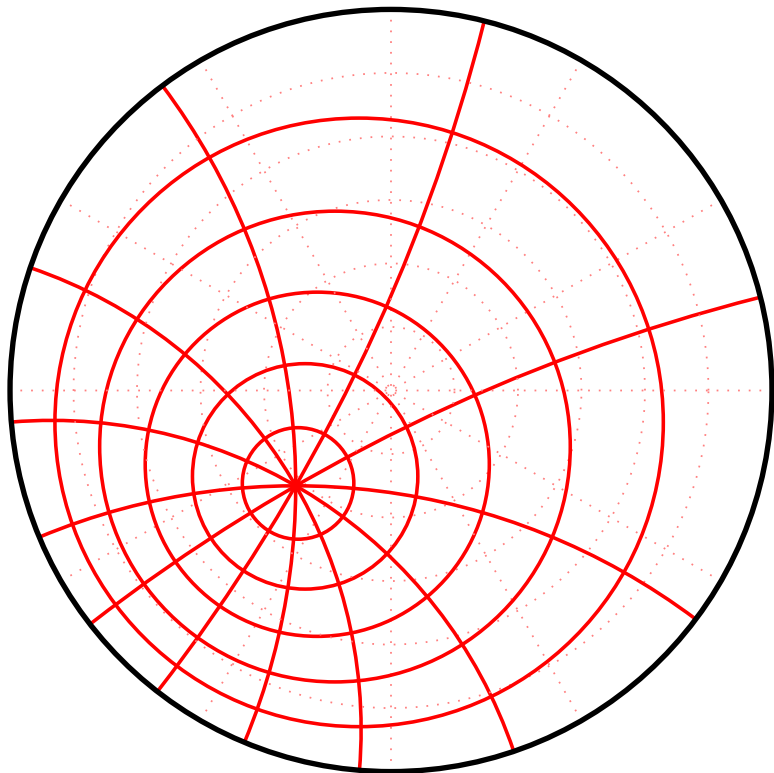


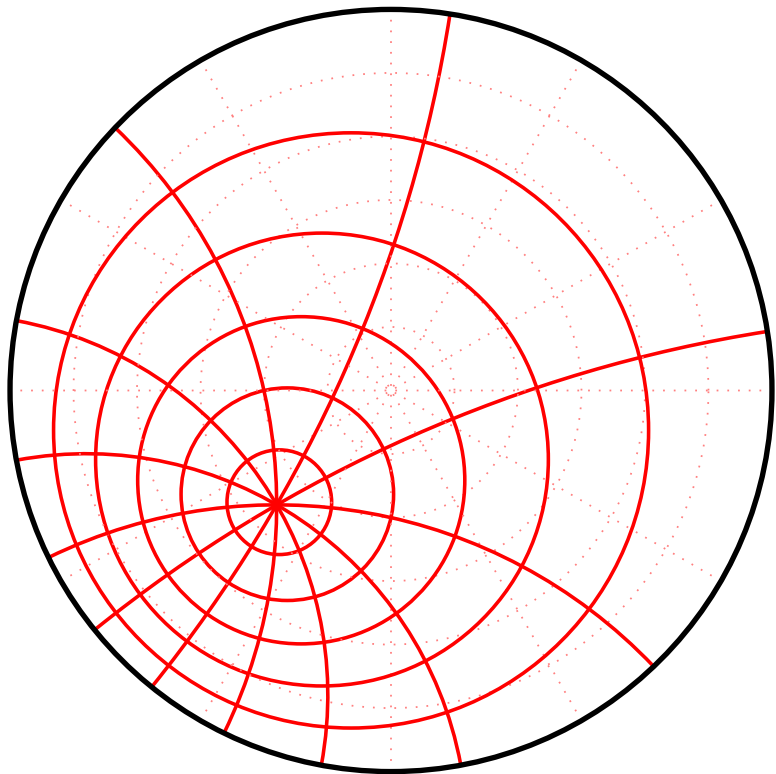


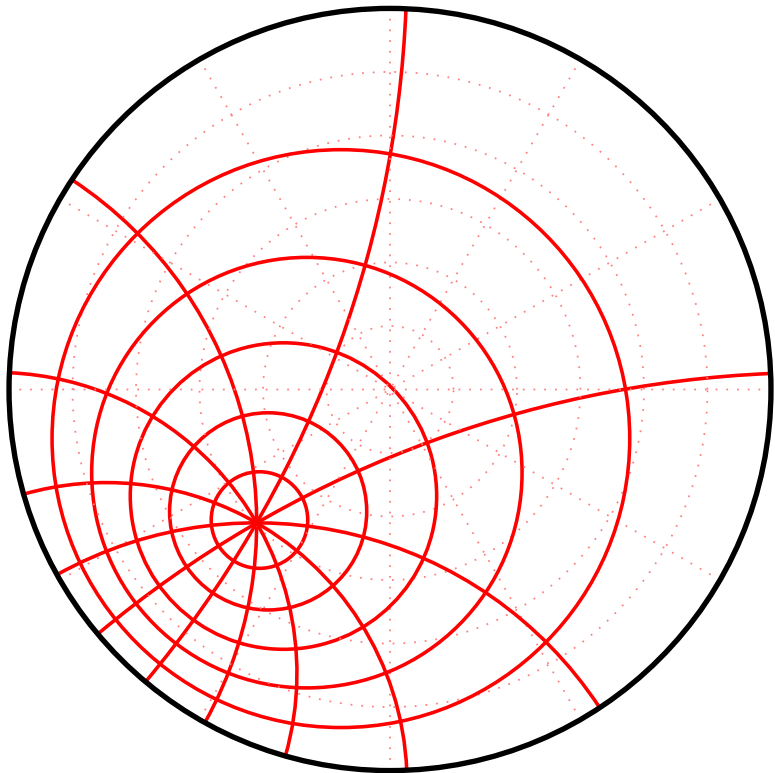


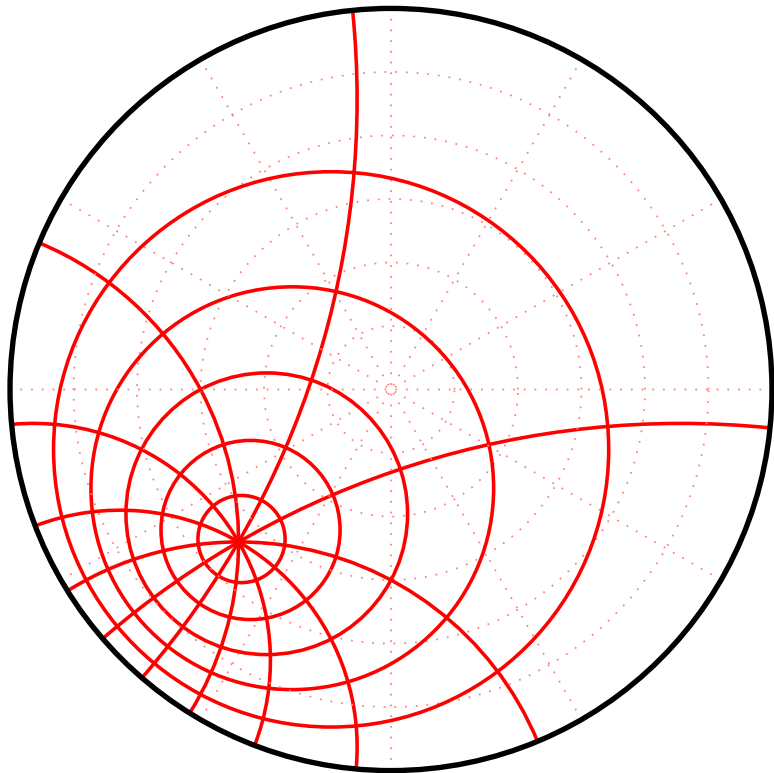


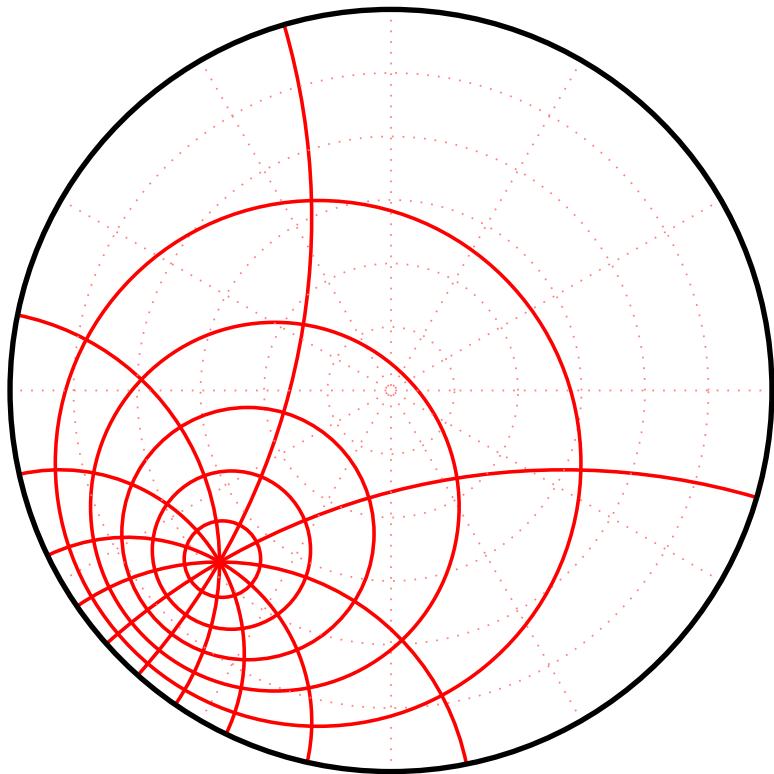


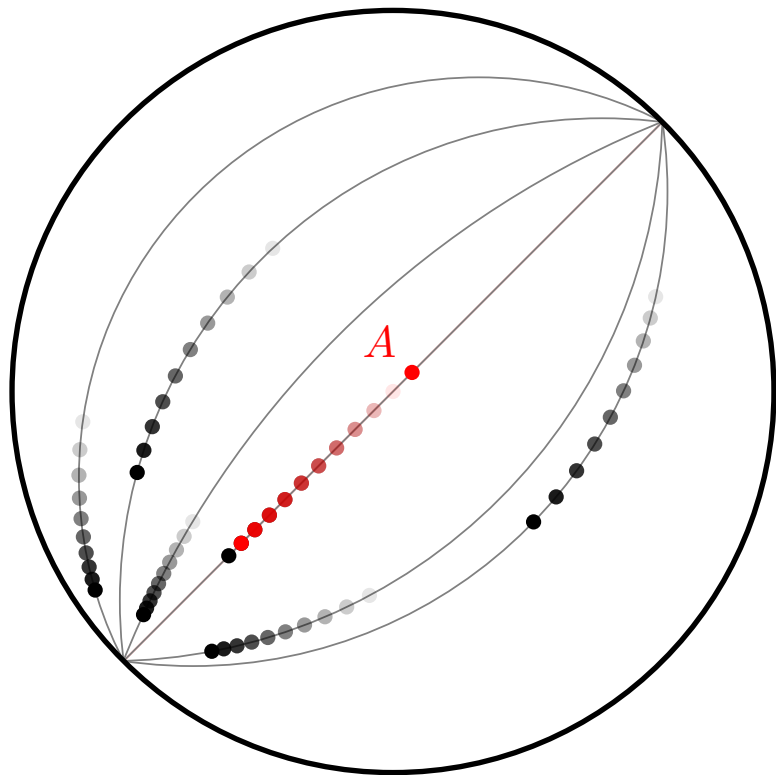






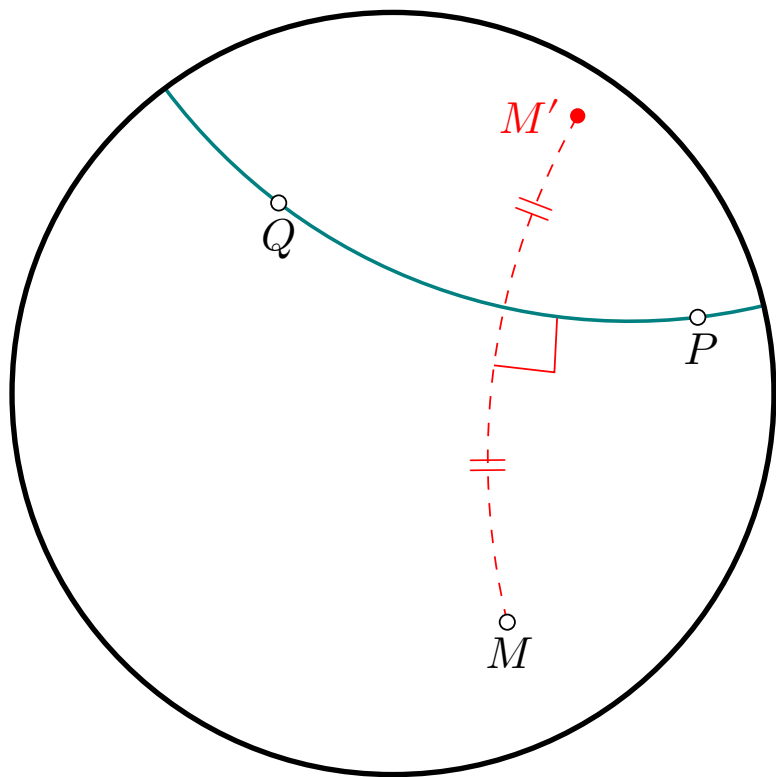


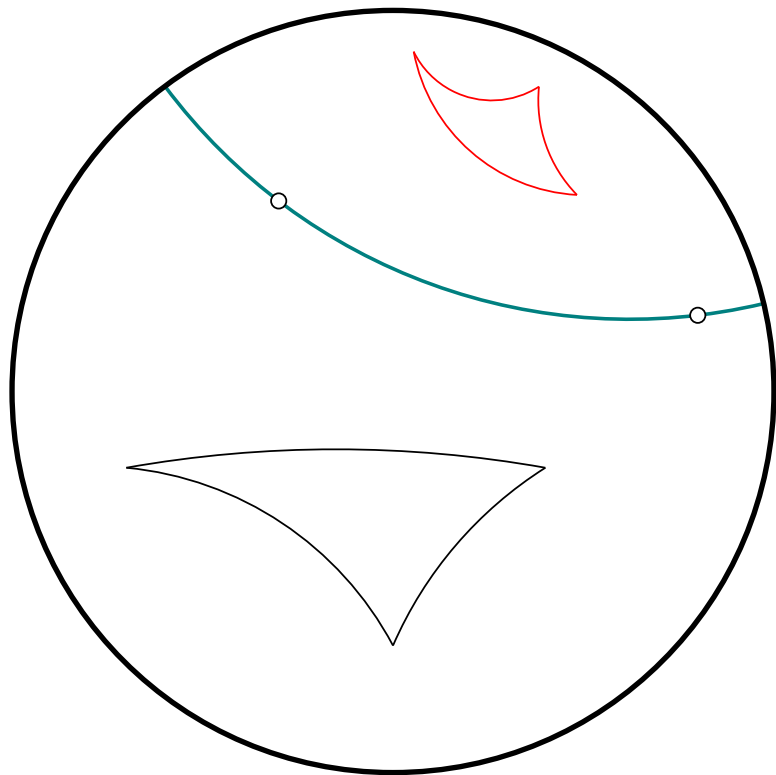




**Automorphismes paraboliques
(NON)**

Symétries axiales





Partie II

Programmer en Lua

dans un document (Lua)LaTeX

**Compilation avec lualatex
(plutôt que pdflatex)**

Très simple : `lualatex myfile.tex`, ou `latexmk --lualatex myfile.tex`.

Preamble minimal :

```
\documentclass{article}
\usepackage[french]{babel}
\usepackage{mathtools} % à charger avant unicode-math
\usepackage{unicode-math} % charge fontspec
```

Avantage immédiat (?) de la compilation avec LuaLaTeX choix de n'importe quelle police :

```
\setmainfont{Comic Sans MS}
```

Quelle indignité...

```
\setmainfont{Papyrus}
```

Je vous demande de vous arrêter !

Pour plus d'informations sur les polices disponibles, voir les documents récents de Daniel Flipo :

— <https://danielflipo.fr/doc/luatex/pdf2lua.pdf>

— <https://danielflipo.fr/doc/luatex/fonts.pdf>

**Lua, un langage minimaliste
(Mais efficace !)**

- Très simple à apprendre
- Syntaxe très proche de Python (avec des `end` pour fermer les blocs)
- site web : lua.org
- Manuel : <https://www.lua.org/manual/>
- Livre : *Programming in Lua* (5ème édition, 1ère édition gratuite sur <https://www.lua.org/pil/contents.html>)
- Notes de cours : <https://ic.ufrj.br/~fabiom/lua/>
- Utilisé dans un nombre croissant de packages latex, par exemple l'excellent `tkz-euclide`, pour augmenter la rapidité et la précision des calculs.

Utilisation de Lua dans un document LaTeX

La factorielle de 5 vaut

```
\directlua{
  local result = 1
  for k=1,5 do result = result * k end
  tex.sprint(result)}.\par
```

La double factorielle de 5 ,
notée $5!!$, est égale à

```
\directlua{local n, result = 5, 1
  while n>0 do
    result = result * n
    n = n-2
  end
  tex.sprint(result)}.\par
```

La puissance de 2 immédiatement
(strict^o) supérieure à 100 est

```
\directlua{local n=1
  while n <= 100 do n = n * 2 end
  tex.sprint(n)}.
```

La factorielle de 5 vaut 120 .

La double factorielle de 5 , notée $5!!$,
est égale à 15 .

La puissance de 2 immédiatement
(strict^o) supérieure à 100 est 128 .

Partie III

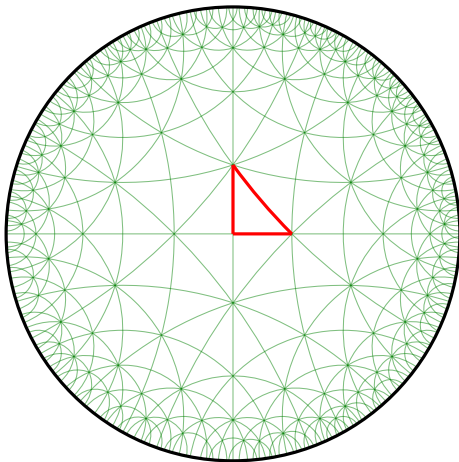
Quelques fonctionnalités du package ‘luahyperbolic’

Informations pratiques et liens

— Page CTAN : <https://ctan.org/pkg/luahyperbolic>

— Repo github : <https://github.com/dmegy/luahyperbolic>

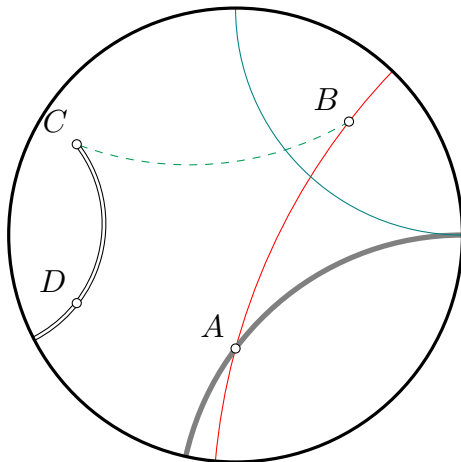
Attention, la version sur github est la plus à jour si l'on désire tester de nouvelles fonctionnalités.



```

\begin{luacode*}
local A = -complex.I/2
local B = (1+complex.I)/2
local C = complex(-0.7,0.4)
local D = complex(-0.7,-0.3)
hyper.tikzBegin()
hyper.drawLine(A, B, "red")
hyper.drawSegment(B,C,"dashed, ForestGreen"
    ↪ )
hyper.drawRay(C,D,"double,double distance=1
    ↪ pt")
hyper.drawLine(A,1,"gray,line width=2pt")
hyper.drawLine(1, complex.I, "teal")
hyper.drawPoints(A, B, C, D)
hyper.labelPoints(A, "$A$", B, "$B$", C, "
    ↪ $C$", D, "$D$")
hyper.tikzEnd()
\end{luacode*}

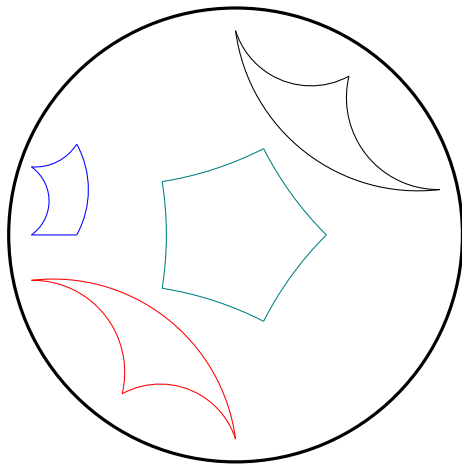
```



```

\begin{luacode*}
local A = complex(.9,.2)
local B = complex(.5,.7)
local C = complex(0,.9)
hyper.tikzBegin()
hyper.drawTriangle(A, B, C)
hyper.drawTriangle(-A, -B, -C, "red")
hyper.drawPolygon(-.9, -.7, complex(-.7,.4)
    ↪ , complex(-.9,.3), "blue")
local points={}
for k=1,5 do
    table.insert(points, complex.polar(.4,k
        ↪ *2*math.pi/5))
end
hyper.drawPolygonFromTable(points, "teal")
hyper.tikzEnd()
\end{luacode*}

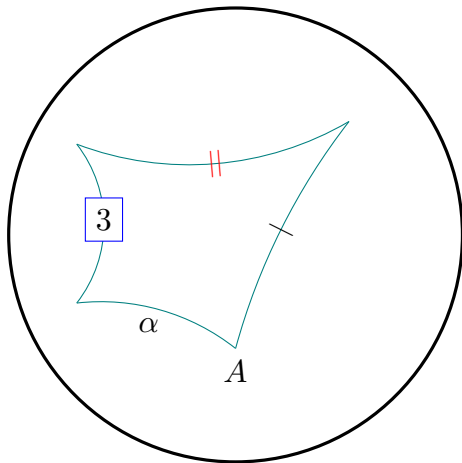
```



```

\begin{luacode*}
local A = -complex.I/2
local B = (1+complex.I)/2
local C = complex(-0.7,0.4)
local D = complex(-0.7,-0.3)
hyper.tikzBegin()
hyper.drawPolygon(A, B, C, D, "teal")
hyper.markSegment(A,B,"|")
local marking = "\\textcolor{red}{||}"
hyper.markSegment(B,C,marking)
hyper.labelSegment(C,D,"$3$", "fill=white,
    ↪ draw=blue")
hyper.labelSegment(D, A,"$\\alpha$", "below
    ↪ ")
hyper.labelPoint(A, "$A$", "below")
hyper.tikzEnd()
\end{luacode*}

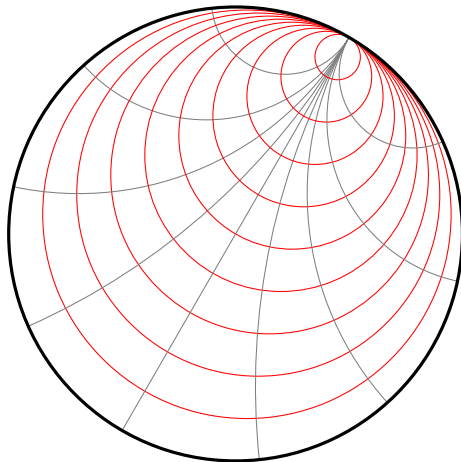
```



```

\begin{luacode*}
local theta=math.pi/3
local idealPoint = complex.exp_i(theta)
local N = 10
hyper.tikzBegin()
for k=1,N-1 do
  local point = idealPoint * (2*k/N-1)
  hyper.drawHorocycle(idealPoint,point,"red
    ↪ ")
  -- and an orthogonal geodesic :
  local endPoint = complex.exp_i(theta+2*k*
    ↪ math.pi/N)
  hyper.drawLine(idealPoint,endPoint,"gray"
    ↪ )
end
hyper.tikzEnd()
\end{luacode*}

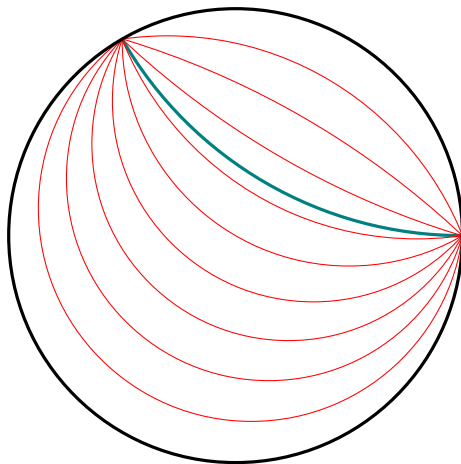
```



```

\begin{luacode*}
hyper.tikzBegin()
local A = 1
local B = complex.J
hyper.drawLine(A,B,"very thick, teal")
local N = 10
for k=1,N-1 do
    local P = (2*k/N-1)*(-B^2)
    hyper.drawHypercycleThrough(P, A, B, "red
        ↪ ")
end
hyper.tikzEnd()
\end{luacode*}

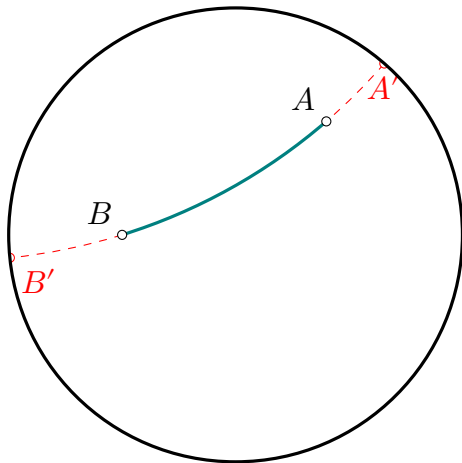
```



```

\begin{luacode*}
local A = complex(.4,.5)
local B = complex(-.5,0)
local AA, BB = hyper.endpoints(A,B)
hyper.tikzBegin()
hyper.drawSegment(A,B, "very thick, teal")
hyper.drawSegments(AA,A, B, BB, "dashed,
    ↪ red")
hyper.labelPoints(A, "$A$", B, "$B$")
hyper.labelPoint(AA, "$A'$", "red, below")
hyper.labelPoint(BB, "$B'$", "red, below
    ↪ right")
hyper.drawPoints(A, B)
hyper.drawPoints(AA,BB,"white, draw=red")
hyper.tikzEnd()
\end{luacode*}

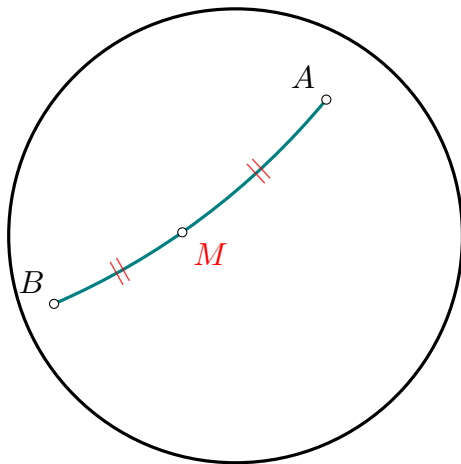
```



```

\begin{luacode*}
local A = complex(0.4,0.6)
local B = complex(-0.8,-0.3)
local M = hyper.midpoint(A,B)
hyper.tikzBegin()
hyper.drawSegment(A,B, "very thick, teal")
hyper.drawPoints(A, B, M)
hyper.labelPoints(A, "$A$", B, "$B$")
hyper.labelPoint(M, "$M$", "below right,
    ↪ red")
local marking = "\\textcolor{red}{||}"
hyper.markSegment(A,M,marking)
hyper.markSegment(M,B,marking)
hyper.tikzEnd()
\end{luacode*}

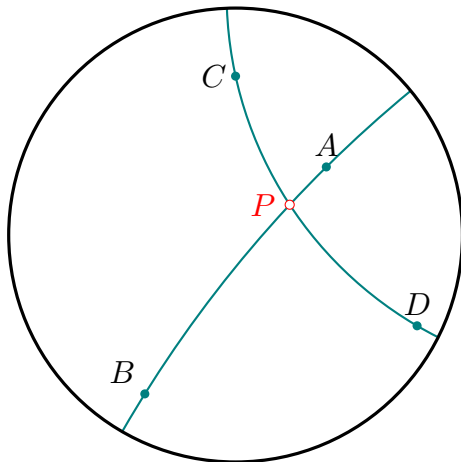
```




```

\begin{luacode*}
local A = complex(0.4,0.3)
local B = complex(-0.4,-0.7)
local C = complex(0,0.7)
local D = complex(0.8,-0.4)
local P = hyper.interLL(A, B, C, D)
hyper.tikzBegin()
hyper.drawLines(A, B, C, D, "thick, teal")
hyper.drawPoints(A, B, C, D, "thick, teal")
hyper.drawPoint(P, "white, draw=red")
hyper.labelPoint(A, "$A$", "above")
hyper.labelPoint(B, "$B$")
hyper.labelPoint(C, "$C$", "left")
hyper.labelPoint(D, "$D$", "above")
hyper.labelPoint(P, "$P$", "left=.1cm, red"
  ↪ )
hyper.tikzEnd()
\end{luacode*}

```

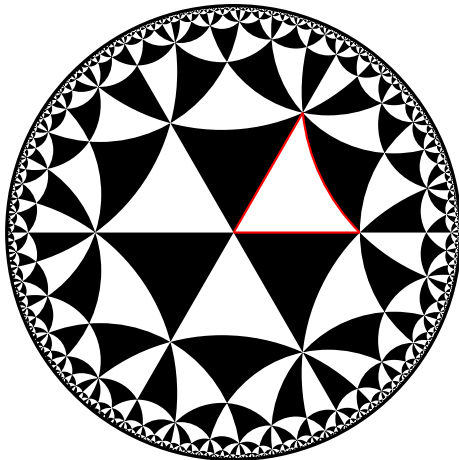


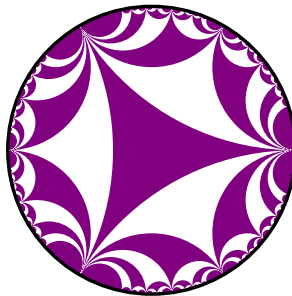
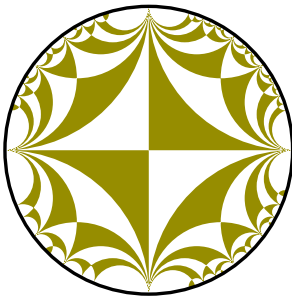
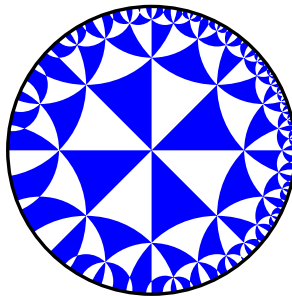
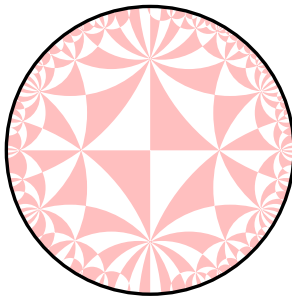
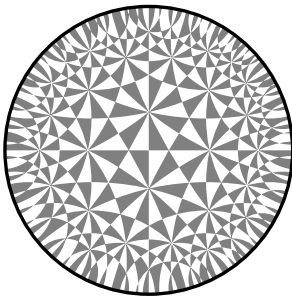
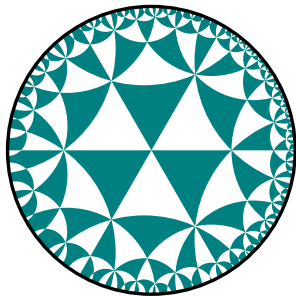
Pavages hyperboliques

```

\begin{luacode*}
local PI = math.pi
local DEPTH = 10
local A, B, C = hyper.triangleWithAngles(PI
    ↪ /3, PI/4, PI/5)
hyper.tikzBegin()
hyper.fillTilingFromTriangle(A, B, C, DEPTH
    ↪ )
-- default color is black
hyper.drawPolygon(A, B, C, "thick,red")
hyper.tikzEnd()
\end{luacode*}

```





Aperçu des fonctions disponibles

Module ``complex'` (petite lib de nombres complexes)

<code>__add</code>	<code>abs</code>	<code>dot</code>	<code>isInteger</code>	<code>nonzero</code>
<code>__div</code>	<code>abs2</code>	<code>exp</code>	<code>isNear</code>	<code>polar</code>
<code>__eq</code>	<code>arg</code>	<code>exp_i</code>	<code>isNearInteger</code>	<code>rotate</code>
<code>__mul</code>	<code>clone</code>	<code>invert</code>	<code>isNot</code>	<code>rotate90</code>
<code>__pow</code>	<code>coerce</code>	<code>isClose</code>	<code>isReal</code>	<code>scal</code>
<code>__sub</code>	<code>conj</code>	<code>isColinear</code>	<code>isUnit</code>	<code>toPolar</code>
<code>__toString</code>	<code>det</code>	<code>isComplex</code>	<code>log</code>	<code>unit</code>
<code>__unm</code>	<code>distinct</code>	<code>isImag</code>	<code>new</code>	

Module ``hyper'` (lib principale)

Attention, les fonctions préfixées par « `_` » vont peut-être redevenir privées donc non disponibles pour l'utilisateur. Il est demandé de ne pas les utiliser.

<code>_assert</code>	<code>_midpoint_to_origin</code>
<code>_assert_in_closed_disk</code>	<code>_on_circle</code>
<code>_assert_in_disk</code>	<code>_on_line</code>
<code>_circle_to_euclidean</code>	<code>_on_segment</code>
<code>_coerce_assert_in_closed_disk</code>	<code>_radial_half</code>
<code>_coerce_assert_in_disk</code>	<code>_radial_scale</code>
<code>_ensure_order</code>	<code>_warning</code>
<code>_error</code>	<code>angle</code>
<code>_geodesic_data</code>	<code>automorphism</code>
<code>_in_closed_disk</code>	<code>automorphismFromPairs</code>
<code>_in_disk</code>	<code>automorphismSending</code>
<code>_in_half_plane</code>	<code>barycenter2</code>
<code>_metric_factor</code>	<code>distance</code>

distanceToLine
drawAngle
drawAngleBisector
drawArc
drawCircle
drawCircleThrough
drawCircumcircle
drawHorocycle
drawHypercycleThrough
drawIncircle
drawLine
drawLineFromVector
drawLines
drawLinesFromTable
drawPerpendicularBisector
drawPerpendicularThrough
drawPoint

drawPointOrbit
drawPoints
drawPolygon
drawPolygonFromTable
drawPolyline
drawPolylineFromTable
drawRay
drawRayFromPoints
drawRayFromVector
drawRegularPolygon
drawRightAngle
drawSegment
drawSegments
drawSemicircle
drawTangentAt
drawTilingFromAngles
drawTilingFromTriangle

drawTriangle
drawVector
endpoints
endpointsAngleBisector
endpointsPerpendicularBisector
expMap
fillCircle
fillTilingFromAngles
fillTilingFromTriangle
interCC
interLC
interLL
interpolate
labelPoint
labelPoints
labelSegment
markSegment

markSegments
midpoint
mobiusTransformation
parabolic
pointOrbit
projection
propagateGeodesics
randomPoint
reflection
rotation
rotationFromPair
symmetry
symmetryAroundMidpoint
tangentVector
tikzBegin
tikzClearBuffer
tikzDefineNodes

tikzEnd
tikzExport
tikzGetFirstLines
tikzPrintf
tikz_shape_closed_line
tikz_shape_euclidean_segment
tikz_shape_segment

triangleCentroid
triangleCircumcenter
triangleIncenter
triangleOrthocenter
triangleWithAngles

Merci !

Dernière version et code source des slides :

<https://github.com/dmegy/expose-luahyperbolic-gutenberg-2026>